

T.C.
ERZURUM TEKNİK ÜNİVERSİTESİ
ELEKTRİK ELEKTRONİK MÜHENDİSLİĞİ-BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

TAM OTONOM DONANIM TASARIMI VE RISC-V TABANLI
YONGADA SİSTEM GERÇEKLEMESİ

Taha Tahsin AKYÜZ
Naz Afra ÖKSÜZ
Emine BEKTAŞ

TEMMUZ - 2026

T.C.
ERZURUM TEKNİK ÜNİVERSİTESİ
ELEKTRİK ELEKTRONİK MÜHENDİSLİĞİ-BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

Başlığı: Tam Otonom Donanım Tasarımı ve RISC-V Tabanlı Yongada Sistem Gerçeklemesi

Öğrenciler Taha Tahsin AKYÜZ, BİLGİSAYAR MÜHENDİSLİĞİ
Naz Afra ÖKSÜZ, ELEKTRİK-ELEKTRONİK MÜHENDİSLİĞİ
Emine BEKTAŞ, ELEKTRİK-ELEKTRONİK MÜHENDİSLİĞİ

Proje Danışmanları: Dr. Öğr. Üyesi Latif AKÇAY, BİLGİSAYAR MÜHENDİSLİĞİ
Dr. Öğr. Üyesi Sefa KÜÇÜK, ELEKTRİK-ELEKTRONİK

Teslim Tarihi: 03.07.2026

PROJE KAYNAKLARI
Makine ve Teçhizat: Xilinx Artix-7 tabanlı Basys 3 FPGA Geliştirme Kartı (Fiziksel doğrulama için) , VGA Monitör, Kamera (OV7670). Erzurum Teknik Üniversitesi laboratuvar imkanlarıyla sağlanacaktır. Yazılım Kaynakları: AkyuzIDE (Geliştirilen yerel platform) , Qwen2.5-Coder 14B yerel LLM mimarisi , Xilinx Vivado (Sentez ve simülasyon aracı). Açık kaynaklı kütüphaneler ve akademik lisanslar kanalıyla ücretsiz temin edilecektir.
PROJE GRUBU İŞ PAYLAŞIMI
Proje grubu 3 kişiden oluşmakta olup görev dağılımı şu şekildedir: Taha Tahsin AKYÜZ: AkyuzIDE masaüstü platformunun geliştirilmesi, TÜBİTAK 2209-A başvuru süreçleri ve genel rapor yazımı. Naz Afra ÖKSÜZ: Alternatif modül tasarımlarının yapılması, ve ara sunumların hazırlanması. Emine BEKTAŞ: Alternatif modül tasarımlarının yapılması ve bitirme tezi dokümantasyonunun yazılması.
PROJE PLANI KAYNAKLARI
[1] RISC-V International, "The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA," Document Version 20191213, Dec. 2019. [Online]. Available: https://riscv.org/technical/specifications/ [2] McKinsey & Company, "The state of AI in 2024: Gen AI adoption spurts and starts to generate value," McKinsey QuantumBlack, May 2024. [Online]. Available: https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-state-of-ai [3] Digilent, "Basys 3 FPGA Board Reference Manual," Digilent Inc., Pullman, WA, USA, 2016. [Online]. Available: https://digilent.com/reference/programmable-logic/basys-3/reference-manual [4] YosysHQ, "picorv32: A Size-Optimized RISC-V CPU," GitHub repository, 2020. [Online]. Available: https://github.com/YosysHQ/picorv32

IEEE ETİK KURALLARI

Mesleğime karşı şahsi sorumluluğumu kabul ederek, hizmet ettiğim toplumlara ve üyelerine en yüksek etik ve mesleki davranışta bulunmaya söz verdiğimi ve aşağıdaki etik kurallarını kabul ettiğimi ifade ederim:

1. Kamu güvenliği, sağlığı ve refahı ile uyumlu kararlar vermenin sorumluluğunu kabul etmek ve kamu veya çevreyi tehdit edebilecek faktörleri derhal açıklamak;
2. Mümkün olabilecek çıkar çatışması, ister gerçekten var olması isterse sadece algı olması, durumlarından kaçınmak. Çıkar çatışması olması durumunda, etkilenen taraflara durumu bildirmek;
3. Mevcut verilere dayalı tahminlerde ve fikir beyan etmelerde gerçekçi ve dürüst olmak;
4. Her türlü rüşveti reddetmek;
5. Mütenasip uygulamalarını ve muhtemel sonuçlarını gözeterek teknoloji anlayışını geliştirmek;
6. Teknik yeterliliklerimizi sürdürmek ve geliştirmek, yeterli eğitim veya tecrübe olması veya işin zorluk sınırları ifade edilmesi durumunda ancak başkaları için teknolojik sorumlulukları üstlenmek;
7. Teknik bir çalışma hakkında yansız bir eleştiri için uğraşmak, eleştiriye kabul etmek ve eleştiriye yapmak; hatları kabul etmek ve düzeltmek, diğer katkı sunanların emeklerini ifade etmek;
8. Bütün kişilere adilane davranmak; ırk, din, cinsiyet, yaş, milliyet, cinsi tercih, cinsiyet kimliği veya cinsiyet ifadesi üzerinden ayrımcılık yapma durumuna girişmemek;
9. Yanlış veya kötü amaçlı eylemler sonucu kimsenin yaralanması, mülklerinin zarar görmesi, itibarlarının veya istihdamlarının zedelenmesi durumlarının oluşmasından kaçınmak;
10. Meslektaşlara ve yardımcı personele mesleki gelişimlerinde yardımcı olmak ve onları desteklemek,

IEEE Yönetim Kurulu tarafından Ağustos 1990'da onaylanmıştır.

03.07.2026

Taha Tahsin AKYÜZ

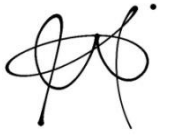
Naz Afra ÖKSÜZ

Emine BEKTAŞ

BİTİRME PROJESİ GÖREV DAĞILIMI VE KATKI ORANI BEYANI

Erzurum Teknik Üniversitesi Bilgisayar Mühendisliği-Elektrik Elektronik Mühendisliği Bölümü bitirme projesi yazım kurallarına uygun olarak hazırladığım “Tam Otonom Donanım Tasarımı ve RISC-V Tabanlı Yongada Sistem Gerçeklemesi” Başlıklı proje dokümanımın içindeki bütün bilgilerin doğru olduğunu, bilgilerin üretilmesi ve sunulmasında bilimsel etik kurallarına uygun davrandığımı, kullandığım bütün kaynakları atıf yaparak belirttiğimi, maddi ve manevi desteği olan tüm kurum/kuruluş ve kişileri belirttiğimi, burada sunduğum veri ve bilgileri unvan almak amacıyla daha önce hiçbir şekilde kullanmadığımı beyan ederim.

Bu projede yer alan görev dağılımı ve katkı oranları, grup üyelerinin ortak beyanıdır. Her bir öğrenci, tabloda belirtilen görevleri fiilen yerine getirdiğini belirtilen katkı oranlarının gerçeği yansıttığını kabul ve beyan eder.

Adı Soyadı	Projede ve Tezde Üstlendiği Görevler	Katkı Oranı (%)	İmza
Taha Tahsin Akyüz	AkyüzIDE Masaüstü Platformu Geliştirme ve TÜBİTAK 2209-A Başvuru ve Rapor Yazımı	%40	
Naz Afra Öksüz	Alternatif Modül Tasarımı ve Ara Sunumlar	%30	
Emine Bektaş	Alternatif Modül Tasarımı ve Bitirme Tezi Yazımı	%30	

Kurallar:

- Toplam katkı %100 olmak zorundadır
- Tüm grup üyeleri tabloyu birlikte doldurur
- Gerçeğe aykırı beyan etik ihlal sayılır

ÖNSÖZ

Modern dijital donanım tasarımı ve işlemci mimarisi geliştirme süreçleri, mühendislik eğitiminin en karmaşık ve yüksek akademik birikim gerektiren alanlarından biridir. RISC-V komut seti mimarisinin (ISA) açık kaynaklı yapısı küresel ölçekte güçlü bir araştırma zemini sunsa da bu yapıları Verilog Donanım Tanımlama Dili (HDL) gibi düşük seviyeli dillerle tasarlamak; saat senkronizasyonu ve yarış durumları gibi dik bir öğrenme eğrisi gerektiren ciddi zorluklar barındırır. Mevcut endüstriyel araçların yapay zekâ entegrasyonundan yoksun olması ve geliştiricileri manuel simülasyon loglarını okumaya mahkum etmesi, bu projenin temel motivasyonunu ve sınırlarını belirlemiştir.

Çalışma boyunca bizlerden yardımlarını esirgemeyen ve çalışmamızı planlamamızda bizlere rehberlik eden değerli danışmanlarımız Dr. Latif Akçay ve Dr. Sefa Küçük' e içtenlikle teşekkür ederiz.

Taha Tahsin AKYÜZ

Naz Afra ÖKSÜZ

Emine BEKTAŞ

ERZURUM, 2026

İÇİNDEKİLER

	Sayfa
ÖNSÖZ.....	vii
İÇİNDEKİLER.....	viii
ÖZET	ix
ŞEKİLLER LİSTESİ.....	x
TABLolar LİSTESİ	xi
SİMGELER.....	xii
KISALTMALAR.....	xiii
1. GİRİŞ.....	1
2. YAPAY ZEKÂ.....	2
3. AMAÇ VE HEDEFLER.....	3
4. ÖZGÜN DEĞER.....	4
5. LİTERATÜR TARAMASI	6
6. RISC-V KOMUT SETİ MİMARİSİ (ISA)	7
7. SİSTEM BİLEŞENLERİ VE TASARIM AKIŞI	8
8. YÖNTEM.....	11
9. UYGULAMA	19
10.AJAN MOTORU	24
11.İŞLEMCI GELİŞTİRME SÜRECİ.....	28
12.SONUÇLAR	32
KAYNAKLAR	33

ÖZET

Tam Otonom Donanım Tasarımı ve RISC-V Tabanlı Yongada Sistem Gerçekleşmesi

Taha Tahsin AKYÜZ

Naz Afra ÖKSÜZ

Emine BEKTAŞ

ERZURUM TEKNİK ÜNİVERSİTESİ

Bilgisayar Mühendisliği-Elektrik Elektronik Mühendisliği Bölümü

Zorlu öğrenme süreçleri, karmaşık hata ayıklama aşamaları ve çoklu yazılım bağımlılıkları, günümüz dijital donanım üretiminde ciddi verimlilik kayıplarına neden olmaktadır. Yürütülen mühendislik tasarımı çalışmasında; geliştiricilerin donanım tanımlama dillerini veya endüstriyel sentez araçlarını derinlemesine bilmesine gerek bırakmayan, süreçleri baştan sona otonomlaştıran yapay zeka tabanlı entegre bir ekosistem modellenmiştir. Platformun işlevsel başarısını kanıtlamak üzere sıfırdan üretilen açık kaynaklı yongada sistem mimarisi ise, önerilen otonom yapının tasarım yetkinliğini ölçen en kapsamlı doğrulama göstergesi olarak konumlandırılmıştır.

Çalışmanın temel hipotezi, donanım dillerine ait özel veri setleriyle ince ayar yapılan yerel büyük dil modellerinin, otonom hata düzeltme adımlarını hızlandıracağı yönündedir. Sistemin ayrıca, derin donanım bilgisi gereksinimini ortadan kaldırarak insan kaynaklı zaman maliyetlerini en aza indireceği öngörülmektedir. Araştırmada, kurgulanan mimarinin üretim kabiliyetini ve doğruluğunu test etmek amacıyla belirlenen on dört farklı senaryo ile iki farklı yapay zeka modeli örneklem olarak seçilmiştir. Yöntem olarak; otonom masaüstü arayüzü yerel dil modeli altyapısıyla kodlanmış, doğrulama amacıyla geliştirilen bütünsel işlemci mimarisi ise on altı bağımsız modül halinde tasarlanmıştır.

Elde edilen bulgular, ekosistemin ürettiği beş aşamalı boru hatlı donanımın tüm test senaryolarını başarıyla tamamladığını ve zamanlama doğruluğunu koruduğunu kanıtlamıştır. Sonuç olarak bu çalışma; yapay zeka entegrasyonu ile donanım üretim süreçlerinin otonom, hızlı, güvenli ve tek bir çatı altında birleştirilebileceğini gösteren yenilikçi bir mühendislik yöntemi sunmaktadır.

Anahtar Kelimeler: Otonom Donanım Tasarımı, Yongada Sistem, İşlemci Tasarımı, Büyük Dil Modelleri, Yapay Zekâ, Entegre Geliştirme Ortamı.

ŞEKİLLER LİSTESİ

	Sayfa
Şekil 1. FPGA Kartı	3
Şekil 2. Yapay Zekâ Destekli Tasarım Akış Diyagramı	10
Şekil 3. Genel Sistem Blok Diyagramı (YZ ile üretilmiştir.)	12
Şekil 4. RV32I Pipeline Veri Yolu Şeması (YZ ile üretilmiştir.)	13
Şekil 5. İnce Ayar Sürecine Ait Terminal Çıktıları ve Donanım Kaynak Kullanımı	17
Şekil 6. Eğitim Kaybı (Training Loss) Grafiği	17
Şekil 7. İnce Ayar Sürecine Ait Değerlendirme (Evaluation) Metriği Grafiği	18
Şekil 8. Eğitim Sürecine Ait TensorBoard Metrik Panosu	18
Şekil 9. AkyuzIDE Kurulumu ve Sistem Araç Taraması	20
Şekil 10. AkyuzIDE Sohbet Paneli Açık Hali	21
Şekil 11. Uygulama Arayüzü	21
Şekil 12. Donanım Doğrulama Sonuçları	23
Şekil 13. AkyuzIDE Ajan Modu	25
Şekil 14. Yürütme Planı ve Onay Ekranı	27
Şekil 15. AkyuzIDE Dağıtım Portalı	27
Şekil 16. AkyuzIDE Yönetici Paneli	28
Şekil 17. VGA Terminal Ana Menüsü	30
Şekil 18. RV32I İşlemcisi Üzerinde Çalışan Snake Oyunu	31
Şekil 19. Kamera Arayüzü	31

TABLÖLAR LİSTESİ

	Sayfa
Tablo 1. Yıllara Göre Yapay Zekâ Benimseme Oranları[31]	2
Tablo 2. Literatürde Öne Çıkan RISC-V İşlemci Mimarileri.....	6
Tablo 3. LLM Kullanım Kısıtları ve Önerilen Çözümler	7
Tablo 4. Mimari Karşılaştırma ve RV32I Avantajları	8

SİMGELER

rd	: Destination Register - İşlem sonucunun yazılacağı donanım yazmacı
rs1	: Source Register 1 - RISC-V komutlarında okunan ilk donanım yazmacı
rs2	: Source Register 2 - RISC-V komutlarında okunan ikinci donanım yazmacı

KISALTMALAR

ALU	: Aritmetik ve Mantık Birimi
API	: Uygulama Programlama Arayüzü (Application Programming Interface)
Ar-Ge	: Araştırma ve Geliştirme
ASCII	: Amerikan Standart Kodlama Sistemi
ASIC	: Uygulamaya Özel Tümleşik Devre
BRAM	: Blok Rastgele Erişimli Bellek (Block RAM)
CMOS	: Tamamlayıcı Metal Oksit Yarı İletken (Complementary Metal-Oxide-Semiconductor)
CPU	: Merkezi İşlem Birimi (Central Processing Unit)
EDA	: Elektronik Tasarım Otomasyonu (Electronic Design Automation)
EX	: Yürütme Aşaması (Execute - Pipeline 3. aşaması)
FPGA	: Alanda Programlanabilir Kapı Dizileri (Field Programmable Gate Array)
FSM	: Sonlu Durum Makinesi (Finite State Machine)
GPIO	: Genel Amaçlı Giriş/Çıkış
GPT	: Üretken Önceden Eğitilmiş Dönüştürücü (Generative Pre-trained Transformer)
HDL	: Donanım Tanımlama Dili (Hardware Description Language)
ID	: Komut Çözümleme Aşaması (Instruction Decode)
IDE	: Entegre Geliştirme Ortamı (Integrated Development Environment)
IF	: Komut Getirme Aşaması (Instruction Fetch)
IP	: Fikri Mülkiyet Çekirdeği (Intellectual Property Core)
IPC	: Döngü Başına Komut Yürütme (Instruction Per Cycle)
ISA	: Komut Seti Mimarisi (Instruction Set Architecture)
JSON	: JavaScript Nesne Notasyonu (JavaScript Object Notation)
LED	: Işık Yayan Diyot (Light Emitting Diode)
LH / LHU	: Load Halfword / Load Halfword Unsigned
LLM	: Büyük Dil Modeli (Large Language Model)
LoRA	: Low-Rank Adaptation
LUT	: Bakış Tablosu (Look-Up Table - FPGA kaynak birimi)
MEM	: Bellek Erişim Aşaması (Memory Access)
MHz	: Megahertz (Frekans Birimi)
MMIO	: Bellek Eşlemeli Giriş/Çıkış (Memory-Mapped I/O)
PC	: Program Sayacı
QLoRA	: Nicelenmiş Düşük Dereceli Uyarlama (Quantized LoRA)
RAW	: Yazma Sonrası Okuma tehlikesi (Read After Write)
RISC-V	: Açık kaynaklı bir komut seti mimarisi
RTL	: Yazmaç Aktarım Seviyesi (Register Transfer Level)
RV32I	: 32-bit Temel Tam Sayı RISC-V Komut Seti Mimarisi
SB / SH / SW	: Store Byte / Store Halfword / Store Word
SCCB	: Seri Kamera Kontrol Veri Yolu (Serial Camera Control Bus)
SFT	: Denetimli İnce Ayar (Supervised Fine-Tuning)
SoC	: Çip Üstü Sistem (System on Chip)
SPI	: Seri Çevre Birimi Arayüzü
SSE	: Sunucu Tarafından Gönderilen Olaylar (Server-Sent Events)
T.C.	: Türkiye Cumhuriyeti

TÜBİTAK: Türkiye Bilimsel ve Teknolojik Araştırma Kurumu
UART : Evrensel Asenkron Alıcı-Verici
VCD : Değer Değişim Dökümü (Value Change Dump)
VGA : Video Grafik Dizisi
VRAM : Video Rastgele Erişimli Bellek (Video RAM / Ekran Kartı Belleği)
WB : Geri Yazma Aşaması (Write Back)
WNS : En Kötü Negatif Süre Farkı (Worst Negative Slack)
YZ : Yapay Zekâ

1. GİRİŞ

Mikroişlemci mimarilerinin tarihi gelişimi incelendiğinde, tasarımların genellikle büyük teknoloji devlerinin mülkiyetinde, kapalı kapılar ardında ve ağır lisans kısıtlamaları altında şekillendiği görülmektedir. Ancak son yıllarda açık kaynaklı ve telif hakkı kısıtlamalarından bağımsız bir yapı sunan RISC-V komut seti mimarisi (ISA), bu geleneksel paradigmayı kökten değiştirmiştir. Modülerliği ve genişletilebilir yapısıyla küresel bir standart haline gelen RISC-V, mikro-mimari yapıyı en alt mantık seviyesine kadar şeffaflaştırma fırsatı tanımasıyla günümüz donanım mühendisliği için eşsiz bir konumdadır.

Buna karşın, bir mimarinin fiziksel bir FPGA veya silikon üzerinde karşılık bulabilmesi için donanım tanımlama dillerinin (HDL) yetkinlikle kullanılması zorunludur. Ancak donanım tasarımı; eşzamanlılık, sinyal gecikmeleri ve yarış durumlarını öngörmeyi gerektiren, öğrenme eğrisi oldukça yüksek bir süreçtir. Günümüz endüstrisindeki ticari araçlar (Vivado, simülatörler vb.) güçlü hesaplama kapasitelerine rağmen hantal ve yapay zekâ entegrasyonlarından uzaktır. Genel amaçlı yapay zekâ asistanları ise donanım dillerinin uzamsal doğası gereği sentezlenemez kodlar üreterek mühendisleri yanıltabilmektedir. Bu durum, entegre devre tasarımı süreçlerinde ciddi zaman ve verimlilik kayıplarına yol açarak alana özelleşmiş, bütünsel bir mühendislik çözümünü zorunlu kılmaktadır.

Söz konusu ihtiyaçları adresleyen bu çalışmanın esas iddiası, dijital donanım tasarımı süreçlerindeki dik öğrenme eğrisini ve araç bağımlılığını ortadan kaldırarak FPGA tabanlı donanım tasarımı tamamen otonom hale getirmektir. Bu doğrultuda kullanıcının, donanım tanımlama dillerini veya ağır sentez yazılımlarını bilmesine gerek kalmadan sadece doğal dil ile tasarım yapabilmesi hedeflenmiştir. Bu bağlamda geliştirilen; işlemci çekirdeği ile çevre birimlerinin birleştirildiği bütünsel yongada sistem (SoC) mimarisi, önerilen otonom ekosistemin kompleks dijital tasarımları ne derece hatasız ve sentezlenebilir üretebildiğini gösteren en kapsamlı doğrulama göstergesi (benchmark) olarak konumlandırılmıştır.

Bu hedefler doğrultusunda yürütülen mühendislik projesinin kapsamı; çoklu ajan motoru barındıran akıllı masaüstü arayüzünün (AkyuzIDE) kodlanmasını, yerel bir dil modelinin donanım dillerine özel veri setleriyle ince ayar (fine-tuning) edilmesini ve kullanıcı kodlarını güvenlik gerekçesiyle yerelde işleyen alternatif bir otomasyon akışının oluşturulmasını içermektedir. Ayrıca, otonom aracın üretkenliğini test etmek amacıyla tasarlanan 16 bağımsız RTL modülüne sahip beş aşamalı boru hatlı (pipeline) RV32I işlemci çekirdeği, buna entegre çevre birimleri (UART, SPI, VGA, GPIO vb.) ve sistemin doğruluğunu sınavan 14 farklı testbench senaryosu projenin deneysel kapsamını oluşturmaktadır.

2. YAPAY ZEKÂ

Yapay zekâ teknolojileri, son yıllarda büyük dil modellerinin (LLM) yükselişiyle birlikte teknik dünyada devrimsel bir dönüşüm başlatarak modern mühendislik süreçlerinin vazgeçilmez bir parçası haline gelmiştir. Bu sistemler; kod üretimi, test otomasyonu ve hata analizi gibi karmaşık aşamaları otomatikleştirip hızlandırarak süreçlerde ciddi zaman ve verimlilik artışı sağlamaktadır. Geliştirme döngülerini kısaltarak yüksek maliyet bariyerlerini düşüren yapay zekâ asistanlığı, veriye dayalı çözüm önerileriyle hem hata payını en aza indirmekte hem de tasarım süreçlerini çok daha erişilebilir, ekonomik ve sürdürülebilir bir yapıya kavuşturmaktadır.

2017 yılından 2025 yılına kadar olan genel YZ benimsenme oranları **Tablo 1** olarak sunulmuştur.

Tablo 1. Yıllara Göre Yapay Zekâ Benimseme Oranları[31]

Yıl	Kurumsal YZ Kullanım Oranı (%)	Önemli Gelişmeler
2017	%20	Yapay zekânın kurumsal süreçlere ilk entegrasyonu.
2018	%47	Benimsenme oranında %100'den fazla artış yaşandı.
2019	%58	İlk büyük büyüme dalgasının zirvesi.
2020	%50	Pandemi sürecinde stratejik odak değişiklikleri nedeniyle hafif düşüş.
2021	%56	YZ yatırımları bir önceki yıla göre iki katına çıktı.
2022	%50	Kullanım oranları bir süreliğine stabilize oldu.
2023	%55	Üretken YZ'nin ortaya çıkışıyla yeni bir ivme başladı.
2024	%72	ChatGPT gibi araçların yaygınlaşmasıyla rekor artış.
2025	%78	Tahmin edilen ve güncel raporlarda bildirilen en yüksek seviye.

2.1. Yapay Zekâyı Yönlendirme ve Doğru Komut Verme

Yapay zekâdan doğru sonuç almak için "İstem Mühendisliği" (Prompt Engineering) prensiplerine uygun net komutlar verilmelidir. Bu süreçte modele bir uzmanlık rolü tanımlamak, konunun bağlamını detaylandırmak ve istenen çıktı formatını açıkça belirtmek temel kriterlerdir. Belirsiz ifadeler yerine adım adım açıklanmış, hedef kitlesi ve sınırları belli olan yönergeler kullanmak, modelin hata payını düşürerek akademik veya teknik beklentilere en uygun cevabı üretmesini sağlar. Sürecin başarısı büyük oranda doğru kelimelerin seçimine ve komutların

yapılandırılmasına bağlıdır; çünkü yapay zekaya ne kadar net, spesifik ve bağlamsal detaylar içeren bir kılavuz sunulursa, sistem o derece yüksek doğrulukta bir içerik üretecektir[31].

2.2. Büyük Dil Modelleri (LLM) ve Donanım Tanım Dilleri (HDL) İlişkisi

Büyük dil modelleri (LLM), günümüzde Python veya C++ gibi yazılım dillerinde oldukça başarılı kodlar yazabilmektedir. Çünkü bu dillerde kodlar, yukarıdan aşağıya doğru sırayla (ardışık olarak) çalışır ve yapay zekâ bu mantığı kolayca kavrar. Ancak iş Verilog gibi donanım tanım dillerine (HDL) geldiğinde durum tamamen değişir. Donanım dilleri bir bilgisayar programı değil; çip üzerindeki elektrik devrelerinin aynı anda (eşzamanlı olarak) nasıl çalışacağını belirleyen birer mimari plandır. Genel amaçlı yapay zekâ modelleri bu uzamsal ve eşzamanlı çalışma mantığını anlayamadığı için, kağıt üstünde doğru görünen ama gerçek bir çipe veya FPGA'e yüklenmeye çalışıldığında hata veren, sentezlenemez kodlar üretir. Bu durum, donanım mühendisliğinin katı kurallarını ve işlemci mimarilerini (RISC-V gibi) hatasız bir şekilde algılayabilecek, bu alana özel olarak eğitilmiş yapay zekâ çözümlerine ihtiyaç duyulduğunu açıkça göstermektedir. Geliştirilen RISC-V işlemci mimarisinin donanımsal doğrulaması ve fiziksel testleri için **Şekil 1**'de ön ve arka yüzü gösterilen Basys 3 FPGA geliştirme kartı kullanılmıştır.



Şekil 1. FPGA Kartı

3. AMAÇ VE HEDEFLER

Bu projenin temel amacı; kodlama süreçlerine özel olarak optimize edilmiş yerel bir yapay zekâ modeli ve özgün bir entegre geliştirme ortamı aracılığıyla, karmaşık donanım tasarım

döngülerini daha güvenli, otonom ve verimli hâle getirmektir. Bu doğrultuda projenin amaç ve hedefleri aşağıda maddeler halinde sıralanmıştır.

Amaçlar:

- ❖ FPGA Tabanlı Donanım Tasarım Süreçlerini Tam Otonom Hale Getirmek
- ❖ HDL ve Endüstriyel EDA Araçlarına (Vivado vb.) Olan Bağımlılığı Yapay Zekâ ile Ortadan Kaldırmak
- ❖ Donanım Sentezlenebilirliğini ve Hata Ayıklamayı Otonom Döngülerle Çözmek
- ❖ Güvenli, Yerel ve Offline Bir Geliştirme Ekosistemi İnşa Etmek

Hedefler:

- ❖ Çoklu Ajan Motoruna Sahip Masaüstü Platformu Geliştirmek
- ❖ Yerel LLM Modellerini Donanım Tasarımına Özel Veri Setleriyle İnce Ayarlamak (Fine-Tune)
- ❖ Geliştirilen Aracın Yetkinliğini Göstermek Üzere Sıfırdan RV32I Çekirdeği Tasarlatmak
- ❖ İşlemci Çekirdeğini Çevre Birimleriyle Donatarak Kompleks Bir Yongada Sistem (SoC) Gerçeklemek
- ❖ FPGA Üzerinde Fiziksel ve Fonksiyonel Doğrulama Yapmak
- ❖ Geliştirilen Sistemi Açık Kaynak Olarak Sunmak

4. ÖZGÜN DEĞER

Bu projenin temel özgün değeri; kodlama süreçlerine özel olarak optimize edilmiş yerel bir yapay zekâ modelinin, geliştirilen entegre masaüstü platformu üzerinden, açık kaynaklı RISC-V mimarisine sahip 5 aşamalı boru hatlı (pipeline) bir işlemci tasarımına ve bu sistemin FPGA donanımı üzerinde uçtan uca doğrulanmasına hizmet etmesidir.

4.1. FPGA ile Donanımsal ve Esnek Doğrulama

İşlemci tasarımı sadece yazılımsal simülasyonda bırakılmamış, Xilinx Artix-7 tabanlı Basys-3 geliştirme kartı üzerinde fiziksel olarak doğrulanmıştır. Platform olarak FPGA'in

seçilmesi literatürdeki şu kritik avantaja dayanmaktadır: Bir FPGA'in tanımlayıcı özelliği, işlevinin üretildikten sonra değiştirilebilmesidir. Bu, bir FPGA üzerinde bir devre uygulayabileceğiniz, test edebileceğiniz ve doğru çalışmazsa değiştirebileceğiniz anlamına gelir[32]. Bu esneklik, yapay zekâ destekli tasarımların yüksek ASIC üretim maliyetlerine katlanılmadan donanım üzerinde gerçek zamanlı ve güvenle test edilmesini sağlamıştır.

4.2. Kodlamaya Özelleştirilmiş Yerel Yapay Zekâ Model Entegrasyonu

Bulut tabanlı genel amaçlı modellerin (ChatGPT, Gemini vb.) aksine, bu projede veri güvenliğini sağlamak amacıyla tamamen yerel (offline) çalışan Qwen2.5-Coder 14B mimarisi kullanılmıştır. Bu model; FPGA ve RISC-V verileriyle özel olarak eğitilmiş olup, donanım sentezleme kurallarına tam uyumlu Verilog HDL kodları üretebilmektedir.

4.3. 5 Aşamalı Boru Hatlı (Pipeline) İşlemci Mimarisi

Akademik çalışmalarda sıklıkla tercih edilen tek çevrimli (single-cycle) modeller yerine, işlem hacmini en üst düzeye çıkaran 5 aşamalı (IF, ID, EX, MEM, WB) bir boru hatlı yapı tasarlanmıştır. Bu modüler akış kurulurken, literatürdeki "Bir boru hattı, bir elemanın çıktısının bir sonrakinin girdisi olduğu, seri olarak bağlanmış veri işleme elemanları koleksiyonudur"[33]. tanımı ve "Borulama mantığında amaç, bir buyruğun bitmesini beklemeden arkasından gelen buyruğu işleme sokmaktır"[34]. yaklaşımı esas alınmıştır.

4.4. Masaüstü Platformu ve Otonom Hata Düzeltme Döngüsü

Tasarım süreçlerini basitleştiren hepsi bir arada entegre masaüstü platformu geliştirilmiştir. Derleyiciden gelen sentaks hataları, zamanlama ihlalleri veya veri tehlikeleri, kullanıcı ile yerel dil modeli arasında kurulan otonom döngü sayesinde gerçek zamanlı analiz edilerek otomatik olarak düzeltilmektedir.

4.5. Zengin Çevre Birimleri ile Gelişmiş Sistem (SoC) Modeli

Tasarım yalın bir işlemci çekirdeğiyle sınırlı kalmayıp; UART, SPI, GPIO gibi endüstriyel standartların yanı sıra kamera arayüzü, bootloader birimleri, VGA terminal ve ekran denetleyicileri gibi özelleştirilmiş çevre birimleriyle donatılarak kapsamlı bir çip üstü sistem (SoC) modeline dönüştürülmüştür.

4.6. Yaygın Etki ve Yeniden Üretilirlik

Projenin tüm kodları ve mimari şemaları GitHub üzerinden açık kaynaklı olarak paylaşılacaktır. Geliştirilen AkyuzIDE platformu ve modüler RISC-V işlemci prototipi, bilgisayar mimarisi ve dijital donanım tasarımı eğitimi alan araştırmacılar ve öğrenciler için tamamen şeffaf ve erişilebilir bir laboratuvar materyali sunarak akademik yaygın etki oluşturmaktadır.

5. LİTERATÜR TARAMASI

Bu bölüm, açık kaynaklı RISC-V işlemci mimarilerinin literatürdeki gelişimini ve 5 aşamalı boru hattı (pipeline) tasarımlarındaki donanımsal çözüm yaklaşımlarını incelemektedir. Ayrıca, güncel üretken yapay zeka modellerinin RTL tasarımı süreçlerindeki mantıksal başarımlar oranları ile endüstrideki yerel (offline) ve güvenli entegrasyon ihtiyaçları literatür verileri ışığında ele alınmaktadır.

5.1. RISC-V İşlemci Mimarisi Literatürü

Bilgisayar mimarisi alanı, MIPS gibi kapalı yapılardan David A. Patterson ve Andrew Waterman öncülüğünde geliştirilen açık kaynaklı RISC-V standardizasyonuna geçişle büyük bir dönüşüm yaşamıştır[2, 3, 8]. Literatürde öne çıkan hazır RISC-V işlemci uygulamaları **Tablo 2**'de sunulmuştur.

Tablo 2. Literatürde Öne Çıkan RISC-V İşlemci Mimarileri

Geliştirici / Mimari	Öne Çıkan Özelliği	Temel Komut Seti
SiFive E Serisi	Gömülü sistem odaklı ticari ve akademik çekirdekler	RV32I / RV64I
PicoRV32 (Clifford Wolf)	Tamamen açık kaynaklı, alan ve boyut optimizasyonlu	RV32I
CVA6 - Ariane (ETH Zurich)	Yüksek performans odaklı, derin boru hattı (pipeline) yapısı	RV64I

Bu hazır IP çekirdekleri çip üzeri sistemlere (SoC) doğrudan entegre edilebilse de, donanım/yazılım ortak tasarımı (co-design) araştırmalarında mimarinin sıfırdan tasarlanması esastır[1, 4, 27]. Literatür, boru hattı tasarımı sırasında oluşan veri bağımlılığı ve kontrol tehlikelerinin donanımsal olarak çözülmesinin, döngü başına komut yürütme (IPC) performansını doğrudan etkilediğini vurgulamaktadır[9, 14, 15].

5.2. Yapay Zekâ Literatürü

Büyük Dil Modellerinin (LLM) kodlama süreçlerine entegrasyonu hızla artsa da, donanım tanımlama dilleri (Verilog, VHDL, Chisel) üzerindeki başarımları yazılım dillerine (C, Python) kıyasla düşüktür. LLM kullanım kısıtları ve önerilen çözümler **Tablo 3**'te verilmiştir.

Tablo 3. LLM Kullanım Kısıtları ve Önerilen Çözümler

Literatürdeki Problem / Kısıt	Sektörel Etkisi	Önerilen Çözüm
Bulut Tabanlı API Bağımlılığı	IP gizliliği endişesi ve askeri projelerde çevrimdışı (offline) çalışma zorunluluğu	Tamamen yerel (local) donanımda çalışan mimari.
Yüksek RTL Hata Oranı	Donanım tasarımında sentaktik ve mantıksal hatalar	RISC-V özelinde eğitilmiş/ince ayar yapılmış model yapısı.
Bütünleşik Araç Eksikliği	Tasarım ve doğrulama süreçlerinin kopuk olması	Arka planda IcarusVerilog ile otonom hata tespiti yapan masaüstü IDE.

6. RISC-V KOMUT SETİ MİMARİSİ (ISA)

Bir işlemci tasarlanırken verilecek en önemli mühendislik kararı, komut seti mimarisinin (ISA) seçimidir. Çünkü bu seçim, işlemcinin ne kadar hızlı çalışacağını, çip üzerinde ne kadar alan kaplayacağını ve hangi yazılımları çalıştırabileceğini doğrudan belirler. İşlemcinin verimli çalışması ise sadece seçilen mimariye değil, bu mimarinin Verilog gibi donanım tanım dilleriyle ne kadar iyi kodlandığına bağlıdır.

6.1. RISC-V Mimarisinin Seçimi ve Önemi

İşlemci tasarımı süreçlerinde kapalı kaynaklı ve lisanslı mimarilerin getirdiği kısıtlamalar, özgün donanım geliştirme önündeki en büyük engeldir. RISC-V, tamamen açık kaynaklı olması sayesinde işlemci mimarisinin en derin noktalarına kadar inilmesine ve modüler bir yapıda özelleştirme yapılmasına imkân tanımaktadır. Proje kapsamında, hedef platform olan Artix-7 FPGA kartı için kaynak kullanımı, performans ve eğitim hedefleri arasında ideal bir denge sunduğu için **RV32I (32-bit Tam Sayı Temel Seti)** tercih edilmiştir. Seçilen RV32I komut setinin adres alanı, yazmaç boyutu ve performans gibi kriterlerde sağladığı kazanımlar **Tablo 4**'te açıkça görülmektedir.

Tablo 4. Mimari Karşılaştırma ve RV32I Avantajları

Özellik	8/16-bit Mimariler	RV32I (Seçilen)
Adres Alanı	Kısıtlı (64KB - 1MB)	Geniş (4GB)
Yazmaç Boyutu	Küçük (8/16-bit)	Yeterli (32-bit)
Performans Potansiyeli	Düşük	Yüksek
Hata Ayıklama	Karmaşık	Yönetilebilir Sinyal Takibi

6.2. İşlemci Veri Yolu ve Ana Modüller

İşlemci çekirdeği, her biri belirli bir fonksiyonu yerine getiren modüllerin uyumlu bir şekilde çalışması esasına dayanmaktadır. İşlemciyi oluşturan ana birimler şu şekilde tanımlanmıştır:

- **Aritmetik ve Mantık Birimi (ALU):** Aritmetik ve mantıksal işlemleri gerçekleştiren ana birimdir.
- **Kontrol Birimi (Control Unit):** Komut alanından gelen bilgilere göre kontrol sinyallerini üreterek veri akışını yönetir.
- **Kayıt Seti (Register File):** Temel veri depolama işlevlerini sağlayan, hızlı erişimli yazmaç grubudur.
- **Program Sayacı (PC):** Program akışını yöneten ve bir sonraki komutun adresini tutan birimdir

6.3. 5-Aşamalı İşlem Hattı (Pipeline) Yapısı

Tek çevrimli (single-cycle) mimarilerin aksine, komut işleme hızını artırmak için 5-aşamalı pipeline yapısı kullanılmıştır. Bu yapı, farklı komutların farklı aşamalarını aynı anda işleterek ideal durumda her saat çevriminde bir komutun tamamlanmasını sağlar.

- 1) **IF (Instruction Fetch):** Komut bellekten getirilir.
- 2) **ID (Instruction Decode):** Komut çözümlenir ve yazmaçlar okunur.
- 3) **EX (Execute):** İşlem ALU' da yürütülür.
- 4) **MEM (Memory Access):** Veri belleğine erişim sağlanır.
- 5) **WB (Write Back):** Sonuç yazmaç setine geri yazılır.

7. SİSTEM BİLEŞENLERİ VE TASARIM AKIŞI

İşlemci tasarım süreci; mimari gereksinimlerin belirlenmesi, donanım tanımlama dilleri (HDL) ile modelleme, simülasyon tabanlı doğrulama ve fiziksel sentezleme aşamalarından oluşan karmaşık bir mühendislik döngüsüdür. Bu bölümde, projenin hayata geçirilmesinde kullanılan donanım/yazılım materyalleri ve yapay zekâ (YZ) destekli hibrit tasarım metodolojisi ayrıntılı olarak ele alınmaktadır.

7.1. Kullanılan Araç ve Teknolojiler

Projenin tasarım, simülasyon ve fiziksel doğrulama aşamalarında kullanılan araçlar; akademik geçerliliği olan, endüstriyel standartları destekleyen ve RV32I mimarisinin gereksinimlerini karşılayan araçlar arasından seçilmiştir

7.1.1. Donanım Materyalleri ve Geliştirme Platformu

Tasarlanan RV32I çekirdeğinin fiziksel doğrulaması ve sentezlenebilirliğinin kanıtlanması için Xilinx Artix-7 (XC7A35T) FPGA çipine sahip Digilent Basys-3 geliştirme kartı birincil donanım platformu olarak kullanılmıştır. Bu platformun seçiminde şu teknik kriterler belirleyici olmuştur:

- **Mimari Uyumluluk:** Artix-7 serisi, 32-bit bir işlemci için gerekli olan mantık hücreleri ve bellek blokları arasında ideal bir denge sunarak kaynak kullanımı ve performans optimizasyonuna olanak tanımaktadır.
- **Giriş/Çıkış Kapasitesi:** İşlemci iç durumlarının, yazmaç değerlerinin ve ALU sonuçlarının anlık takibi için kart üzerindeki 16 anahtar (switch), 16 LED ve 4-digit yedi parçalı gösterge birimleri fiziksel pin eşleşmeleriyle aktif edilmiştir.
- **Verimlilik:** Tasarımın FPGA üzerindeki yerleşimi sonucunda yaklaşık 2,100 LUT kullanılmış, bu da toplam kaynakların sadece %10'una tekabül ederek mimarinin kaynak dostu olduğunu göstermiştir. Söz konusu LUT kullanımı; sentez denemesine, etkinleştirilen çevresel birimlere ve yerleştirme/yönlendirme (place & route) sonuçlarına bağlı olarak denemeden denemeye değişkenlik gösterebilmektedir.

7.1.2. Yazılım Materyalleri ve Geliştirme Araçları

Donanım tanımlama ve doğrulama süreçleri, endüstriyel standarttaki EDA (Electronic Design Automation) araçları ve yenilikçi yazılım entegrasyonları ile yürütülmüştür.

- **Vivado Design Suite (Xilinx):** Verilog ve SystemVerilog kodlarının sentezlenmesi, yerleştirilmesi (implementation) ve FPGA için .bit dosyası üretilmesi süreçlerinde merkezi yazılım olarak kullanılmıştır.
- **Icarus Verilog:** Tasarımın modüler aşamalarında hızlı simülasyon ve terminal tabanlı doğrulama süreçleri için yardımcı araç olarak tercih edilmiştir.
- **Büyük Dil Modelleri (LLM):** Tasarım sürecinde birincil olarak, donanım tanımlama diline göre ince ayarlanmış ve tamamen yerel (çevrimdışı) çalışan akyuz-qwen25-coder modeli kullanılmıştır. Bunun yanı sıra uygulamaya entegre edilen OpenRouter eklentisi sayesinde, kullanıcı tercih ettiğinde GPT-4 ve Claude gibi bulut tabanlı modeller ücretli API üzerinden de devreye alınabilmektedir. Böylece sistem hem veri gizliliği gerektiren durumlarda tümüyle çevrimdışı, hem de daha güçlü muhakeme gerektiğinde çevrim içi olarak esnek biçimde çalışabilmektedir.

7.2. Tasarım ve Uygulama Süreci

Proje, klasik donanım tasarımı yapay zekâ analizleriyle birleştirerek otomatik kod üretimi ve anlık hata teşhisi sağlayan teknoloji odaklı bir iş akışını temel almaktadır. Bu yaklaşım sayesinde, store-after-load gibi karmaşık pipeline hataları saniyeler içinde tespit edilip düzeltilerek tasarım verimliliği artırılmıştır. Geliştirilen bütünleşik arayüz; sohbet tabanlı rehberlik, gerçek zamanlı simülasyon ve optimizasyon süreçlerini tek bir platformda birleştirmektedir. Sürecin tüm detayları adım adım **Şekil 2'**de gösterilmiştir.



Şekil 2. Yapay Zekâ Destekli Tasarım Akış Diyagramı

7.2.1. YZ Destekli Modüler Tasarım Akışı

Proje, geleneksel yöntemleri yapay zekâ hızıyla birleştiren yedi aşamalı bir "Sezgisel İş Akışı" izlemektedir. Süreç, kullanıcının doğal dildeki tasarım gereksinimlerini belirtmesiyle başlar (Örn: "32-bit bir ALU tasarla"). LLM, bu istekleri alarak modüler tasarımı (ALU, Register File, PC vb.) adım adım oluşturur, ardından kod Icarus Verilog ile simüle edilerek terminal çıktıları yine LLM tarafından analiz edilir.

7.2.2. Hata Ayıklama ve İyileştirme Stratejisi

Tasarım sürecindeki en kritik aşama, karmaşık pipeline hatalarının LLM tarafından nanosaniye hassasiyetinde teşhis edilmesidir. Başarısız testlerin waveform (dalga formu) ve log çıktıları LLM'e sunulmuş; model sinyal ilişkilerini analiz ederek sorunun kök nedenini anında tespit etmiştir. Örneğin, bir "store-after-load hazard" durumunda, LLM eksik olan "Store data forwarding" mekanizmasını tespit ederek performansı artıracak kod düzeltmelerini önermiştir.

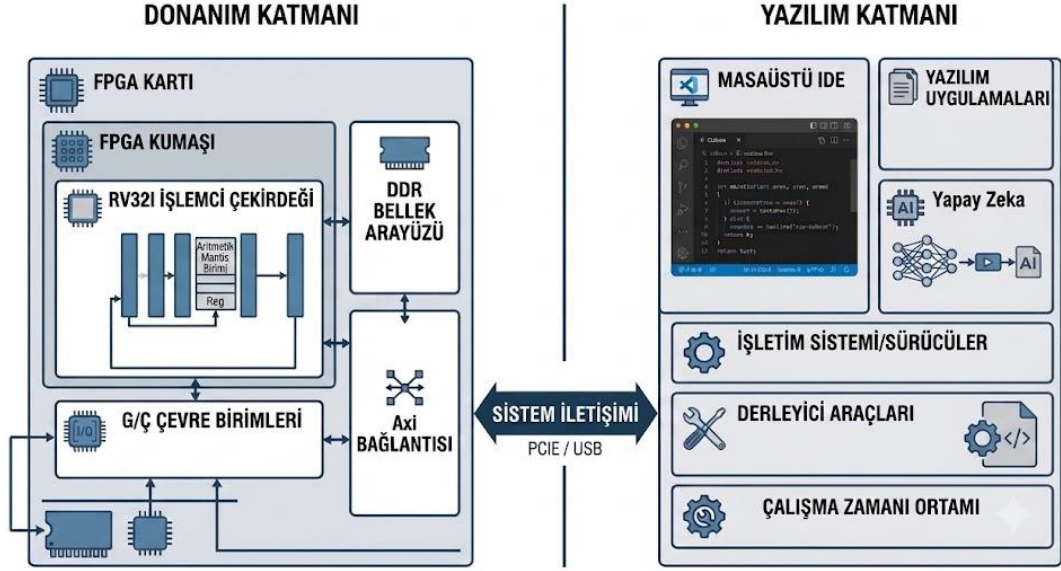
7.2.3. Kademeli Doğrulama Metodolojisi

Doğrulama süreci, sistemin stabilizesini sağlamak amacıyla basitten karmaşığa doğru iki ana aşamada yürütülmüştür:

- **Aşama 1 (tb_quick_test):** ALU operasyonları ve Register File yazma/okuma gibi en temel fonksiyonların doğruluğu test edilmiştir.
- **Aşama 2 (tb_complex):** Mimarının en zorlu yönleri olan veri tehlikeleri (RAW), LoadUse durumları ve dallanma komutlarının yol açtığı kontrol tehlikeleri stres testine tabi tutulmuştur.

8. YÖNTEM

Bu projenin mühendislik yaklaşımı, tek bir sistem kurmak yerine, birbiriyle çift yönlü haberleşme, derleme ve çıktı alışverişi kurabilen tamamen bağımsız ama entegre iki büyük katmana bölünerek tasarlanmıştır. **Şekil 3**'te yapay zekâ ile hazırlanmış genel sistem blok diyagramı verilmiştir.



Şekil 3. Genel Sistem Blok Diyagramı (YZ ile üretilmiştir.)

Donanım katmanı (Hardware Layer), sistemin fiziksel gerçekliğini ve işlem gücünün temelini oluşturur. Bu katmanın merkezinde; tamamen açık kaynak felsefesine dayalı IEEE Verilog HDL standardı ile modüler bir hiyerarşide yazılmış ve FPGA' e sentezlenebilen RV32I işlemci çekirdeği yer alır. Ancak bu çekirdek tek başına yeterli olmadığından; komut ve veri belleklerini, seri iletişim sağlayan evrensel asenkron alıcı-vericiyi (UART) ve kullanıcı girdi/çıktıları için GPIO (LED, buton, switch) birimlerini donanımsal bir adres haritasıyla (Memory Map) birbirine bağlayan kapsamlı bir çip-üstü sistem hiyerarşisi ile bütünleştirilmiştir. Bu yapı, tamamen otonom olarak başlatılabilen (boot) ve dışarıdan aldığı komut dosyalarını işleyebilen bir FPGA prototipi sunmaktadır.

Donanım katmanının hemen üzerine inşa edilen Yazılım katmanı (Software Layer) ise, donanımın sürekli olarak yeni iterasyonlarla yeniden yapılandırılması, sıfırdan Verilog HDL modüllerinin geliştirilmesi, değiştirilmesi, sentaks hatalarından arındırılması ve simülasyon süreçlerinin uçtan uca akıllı bir asistan kontrolünde yönetilmesini sağlayan gelişmiş bir entegre geliştirme platformudur.

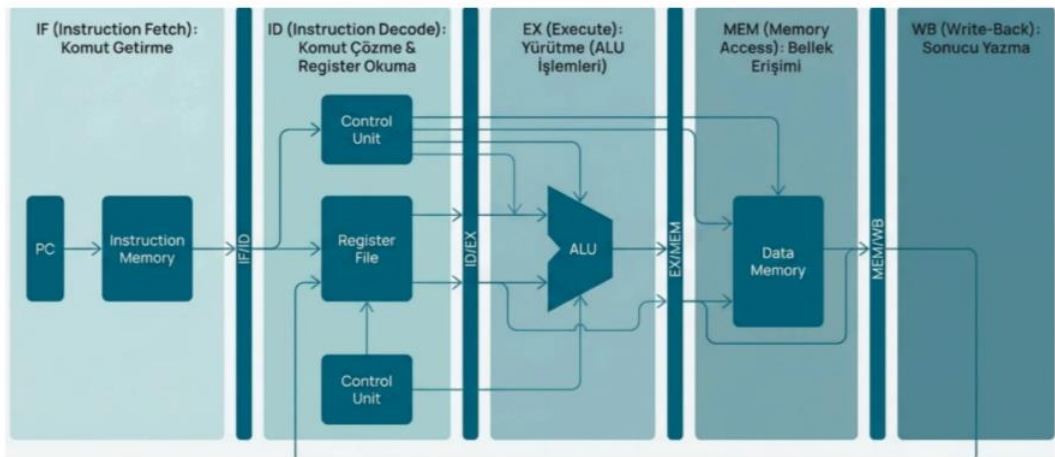
Bu masaüstü platformu, klasik bir metin editörü olmanın ötesinde reaktif bir araçtır. Kullanıcı, grafik ara yüz (GUI) aracılığıyla yeni Verilog işlemci modülleri yazabilir ya da doğrudan entegre yapay zekâ ajanına donanım tanımını doğal dille girerek otonom RTL kodu üretebilir. Üretilen donanım kodu, yazılımın arka plan servisleri vasıtasıyla asenkron olarak IcarusVerilog simülatörüne iletilir.

Eğer simülatör kodda yapısal bir donanım hatası, eşzamanlı atama kısıtı ihlali ya da sinyal boyutlandırma uyumsuzluğu saptarsa, yapay zekâ ajan modülü devreye girer. Ajan, bu

hatayı geliştiriciye hiç yansıtmadan kendi başına okuyarak analiz eder ve akıllı bir düzeltme döngüsü (error loop) başlatır. Özetle; geliştirilen yazılım ortamı donanımın hızla ve hatasız geliştirilmesini sağlayan akıllı bir arayüz sunarken, RV32I işlemci donanımı da üretilen kodların sahada doğrulanması için somut bir test yatağı (testbed) olarak işlev görür.

8.1. RV32I İşlemci Mimarisi

Proje kapsamında tasarlanan çip, 32-bit veri yolu ile 32-bit adres alanına sahip ve RV32I (Base Integer) komut setini bütünüyle işleyebilen modern bir mikroişlemci mimarisidir. Bellek yapısı açısından komut belleği (IMEM) ile veri belleğini (DMEM) mantıksal olarak ayırıştıran Harvard mimarisine dayanan sistem, SoC katmanında ise tekil bir adres uzayına sahip birleşik bir fiziksel veri yolu (UnifiedBus) üzerinde konumlandırılmıştır. İşlemci performansını artırmak ve her saat döngüsünde istikrarlı bir komut yürütme hedefi yakalamak amacıyla 5 aşamalı ardışık boru hattı (pipeline) mimarisi tercih edilmiştir. Mühendislik tasarımı sürecinde kod karmaşasını önlemek ve her birimin bağımsız test edilebilirliğini güvence altına almak adına modüler bir yaklaşım benimsenmiştir; bu doğrultuda sistem, her biri tek bir işlevden sorumlu 16 adet bağımsız Verilog HDL modülü halinde kodlanarak port eşleştirmeleriyle bütünleştirilmiştir. **Şekil 4**'te RV32I pipeline veri yolu şeması ayrıntılı olarak görülmektedir.



Şekil 4. RV32I Pipeline Veri Yolu Şeması (YZ ile üretilmiştir.)

Beş aşamalı bir boru hattı mimarisinde, komutların ardışık olarak bir fabrikadaki bant sistemi gibi akması yüksek bir donanım verimliliği sağlarken, aynı zamanda yazılım komutlarının birbirine olan matematiksel bağımlılıkları nedeniyle mimariyi çökertme potansiyeline sahip, karmaşık "Tehlikeler" (Hazards) mekanizmasını da ortaya çıkarır. İşlemcinin sağlıklı çalışabilmesi ve IPC

kaybının en aza indirilmesi için bu kritik veri ve kontrol durumları, projede özenle tasarlanan şu iki kritik kontrol birimiyle tam kontrol altına alınmıştır:

- **Veri Yönlendirme Birimi:** Ardışık komutların yürütülmesi sırasında, bir komutun ürettiği sonucun henüz kaydedici belleğe yazılmadan bir sonraki komut tarafından aritmetik mantık birimi (ALU) içinde kullanılmak istenmesi durumu, "Yazma Sonrası Okuma" veri tehlikesine yol açar. Bu sorunu çözmek ve işlem akışını duraklatmamak amacıyla bir donanım bloğu tasarlanmıştır. İşlem birimi çıkışında veya bellek aşamasında hazır bekleyen güncel verileri tespit ederek çoklayıcılar yardımıyla doğrudan bir sonraki komutun girişine yönlendirir. Böylece işlemci döngü sürekliliği ve donanım performansı korunmuş olur.
- **Tehlike Tespit Birimi ve Hat Temizleme:** Veri yönlendirme mekanizmasının yetersiz kaldığı tek senaryo, taze verinin ana bellekten ancak 4. aşamada çekilebildiği "Yükleme-Kullanma" tehlikesidir. Bellekten veri okuyan bir komutun hemen ardından bu veriyi işleyecek yeni bir komut geldiğinde, rv32i_hazard birimi hedef ve kaynak kaydediciler arasındaki çakışmayı tespit eder. Bu durumda Program Sayacı dondurulur ve işlem akışına bir döngülük "boş işlem" yerleştirilir. Ayrıca, şartlı atlama komutlarının sonucu 3. aşamada netleştğinde, işlem hattına yanlışlıkla alınmış olan sonraki komutlar donanımsal olarak silinir ve Program Sayacı doğru atlama adresine yönlendirilir.

İşlemcinin tüm parçaları doğrulandıktan sonra, dış dünyayla bağlantısını sağlayan rv32_soc adlı en üst sistem hiyerarşisi (SoC) tasarlanmıştır. Bellek Haritası (Memory Map) adresleme mimarisi, mantıksal olarak komut ve verinin bağımsız işlendiği Harvard esasına dayanırken, FPGA kaynaklarını verimli kullanmak için fiziksel veri yollarında birleşik yapı modeli üzerine kurulmuştur.

8.2. Çevresel Birimler ve Bellek Eşlemeli G/Ç Mimarisi

İşlemci çekirdeği tek başına yeterli olmadığından, gerçek dünya uygulamalarını çalıştırabilmek için MMIO (Bellek Eşlemeli G/Ç) adres kodlama mimarisi üzerine inşa edilmiş zengin bir çevresel birim ekosistemi geliştirilmiştir. Bu yaklaşımda merkezi bir veri yolu yerine adresin üst bitleri ilgili birimi seçen bir kod çözücü (decoder) yapısı kullanılmıştır. İşlemci herhangi bir yükleme ya da depolama komutunu çalıştırdığında, üretilen adres otomatik olarak doğru çevresel birime yönlendirilmektedir.

8.2.1. UART Birimi

Evrensel Asenkron Alıcı-Verici (UART) birimi 115.200 baud hızında çalışmakta olup hem hata ayıklama verilerinin bilgisayar terminaline gönderilmesinde hem de donanımsal önyükleyici (bootloader) aracılığıyla yazılım yüklenmesinde kullanılmaktadır. TX birimi MMIO adres uzayındaki 0x0002_0000 adresine yönelik depolama işlemlerinde tetiklenmekte; RX birimi ise 0x0002_0004 adresinden okunabilmektedir.

8.2.2. GPIO Birimi

Genel Amaçlı Giriş/Çıkış (GPIO) birimi, Basys 3 kartı üzerindeki 16 adet ışık diyotunu (LED), 16 adet anahtar (switch) ve 5 adet düğmeyi (BTNC, BTNU, BTND, BTNL, BTNR) doğrudan işlemciye bağlamaktadır. LED çıkışları 0x0002_0008 adresine yazılarak kontrol edilmekte, anahtar girişleri 0x0002_000C adresinden okunmakta, düğme girişleri ise 0x0002_0010 adresinden erişilebilmektedir. Bu birim, demo uygulamalarında kullanıcı etkileşiminin temel mekanizmasını oluşturmaktadır.

8.2.3. VGA Terminal Birimi

Görsel çıkış altyapısı 80×30 karakter kapasiteli bir metin terminali biçiminde tasarlanmıştır. vga_sync modülü 25 MHz piksel saatiyle 640×480 çözünürlüklü VGA zamanlama sinyallerini üretmektedir. vga_text_ctrl modülü karakter tamponundan (char_buffer) okuduğu ASCII kodlarını, 8×16 piksel boyutundaki font_rom üzerinden piksel verilerine dönüştürmektedir. İşlemci 0x0003_0000 adres aralığına karakter kodu yazarak ekrana metin basabilmektedir. Bunun yanı sıra donanım seviyesinde bir 'Grafik Atlama' (Graphics Bypass) modu eklenmiş; bu mod etkinleştirildiğinde işlemciden bağımsız olarak animasyon üretilmesi mümkün kılınmıştır.

8.2.4. OV7670 Kamera Arayüzü

Prototip 3 ve Prototip 4 kapsamında OV7670 CMOS kamera modülü sisteme entegre edilmiştir. ov7670_sccb_init modülü SCCB protokolüyle kamerayı yapılandırırken, ov7670_capture modülü piksel verilerini çerçeve tamponuna (cam_framebuffer) aktarmaktadır. vga_cam_display modülü ise bu tampondaki görüntüyü VGA çıkışına yönlendirmektedir. Anahtarın (SW15) konumuna göre VGA terminali ile kamera görüntüsü arasında geçiş yapılabilir.

8.2.5. Döngü Sayacı

32-bitlik donanımsal döngü sayacı, sistem saatiyle senkron biçimde her çevrimde bir artmakta ve 0x0002_0014 adresinden okunabilmektedir. Bu birim iki kritik işleve hizmet

etmektedir: birincisi, yazılım zaman gecikmesi (delay) oluşturmak amacıyla mikrosaniye hassasiyetinde zamanlama referansı sağlamak; ikincisi ise oyun uygulamalarında deterministik olmayan bir tohum (seed) değeri üretmek sözde rastgele sayı üreticini başlatmaktır.

8.2.6. Donanımsal Önyükleyici (Bootloader)

hw_bootloader modülü, bitstream yeniden yüklenmeksizin yalnızca yazılımın güncellenmesine olanak tanıyan bir UART tabanlı yükleme mekanizmasıdır. Önyükleyici çalışma mantığı şu adımları izlemektedir: UART alıcısı 'BOOT' sihirli bayt dizisini algıladığında işlemciyi durdurur (halt), ardından gelen ikili veriyi (binary) doğrudan BRAM komut belleğine (IMEM) bayt bayt yazar ve yükleme tamamlandığında program sayacını sıfırlayarak işlemciyi yeni yazılımla yeniden başlatır. Geliştirilen Python yardımcı programı (upload.py) bu süreci otomatize etmekte ve herhangi bir derlenmiş RV32I programını saniyeler içinde kartın üzerine aktarmaktadır.

8.2.7. Wishbone B4 Veri Yolu

MMIO yaklaşımının ötesine geçilerek standart bir veri yolu protokolü olan Wishbone B4 boru hatlı mimarisi uygulanmıştır. Bu protokole STB/CYC/ACK/ERR el sıkışma sinyalleri üzerinden CPU (master) ile çevresel birimler (slave) arasındaki veri transferi yönetilmektedir. wb_interconnect modülü adres kod çözücü ve hakem (arbiter) görevini üstlenirken, wb_slave_wrapper modülü her çevresel birimi Wishbone arayüzüne uyarlamaktadır.

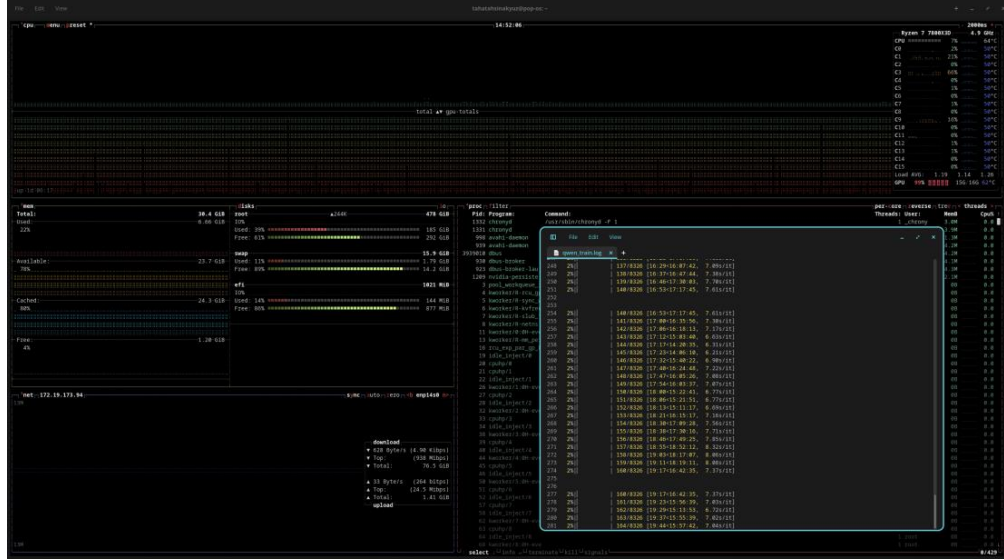
Bu prototipte tam bayt ve yarı bayt erişim genişliği desteği (LB/LBU/LH/LHU/SB/SH/SW) gerçekleştirilmiş ve UART, GPIO, Timer ile SPI birimleri veri yolu üzerine bağlanmıştır.

8.3. Model Eğitimi

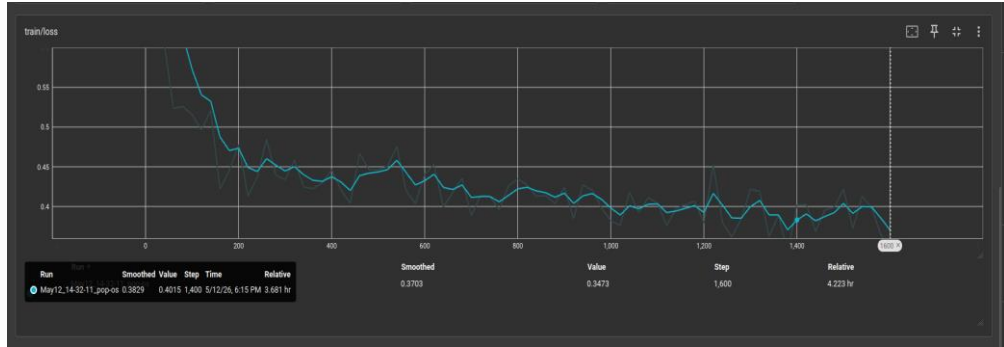
AkyuzIDE' nin yalnızca genel amaçlı büyük dil modelleriyle değil, donanım tanımlama diline özgü ince ayar yapılmış özel bir modelle çalışması, projenin temel özgün değerlerinden birini oluşturmaktadır. Genel amaçlı dil modelleri Python ya da C++ gibi ardışık yürütülen yazılım dillerinde yüksek başarı sergilemekle birlikte, Verilog gibi donanım tanımlama dillerinde ciddi yetersizlikler göstermektedir. Bunun temel nedeni, donanım dillerinin yazılım dillerinden tamamen farklı bir paradigmayı temsil etmesidir: donanım tanımlama dillerinde kod satırları sırayla değil, fiziksel devre kapılarındaki elektrik sinyallerinin eşzamanlı davranışını modelleyecek biçimde çalışır. Bu eşzamanlılık mantığını kavrayamayan modeller, sözdizimsel

olarak doğru görünen ancak gerçek bir yonga ya da FPGA'ya yüklendiğinde hata veren, sentezlenemeyen Verilog kodları üretmektedir.

İnce ayar sürecine ait terminal çıktıları ve donanım logları Şekil 5'te, eğitim adımları ilerledikçe hatanın kararlı şekilde azaldığını gösteren eğitim kaybı (Training Loss) grafiği ise Şekil 6'da verilmiştir.



Şekil 5. İnce Ayar Sürecine Ait Terminal Çıktıları ve Donanım Kaynak Kullanımı



Şekil 6. Eğitim Kaybı (Training Loss) Grafiği

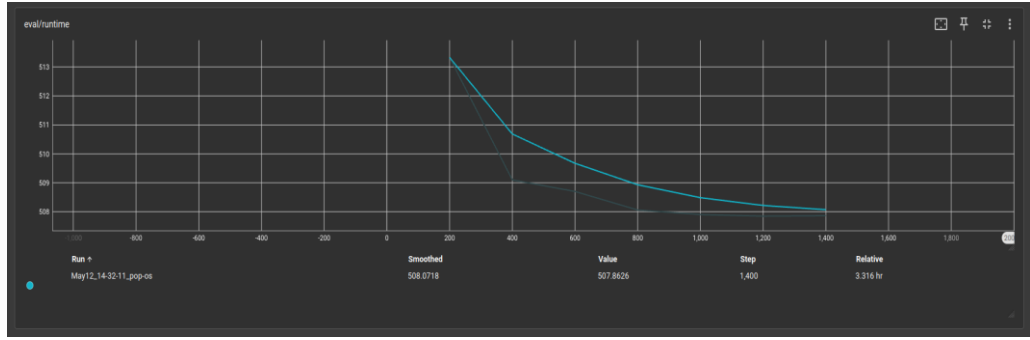
8.3.1. Temel Model Seçimi

İnce ayar için temel model olarak Alibaba Cloud tarafından geliştirilen Qwen2.5-Coder-14B-Instruct mimarisi seçilmiştir. Bu tercihte belirleyici olan etkenler şunlardır: modelin genel kodlama görevlerinde sergilediği yüksek taban performansı, 14 milyar parametrelili yapısının projedeki 16 GB VRAM kapasitesiyle uyumlu olması ve ChatML sohbet şablonunu desteklemesi. Karşılaştırma amacıyla DeepSeek-Coder-V2-Lite-Instruct mimarisi de değerlendirilmiş; ancak bu

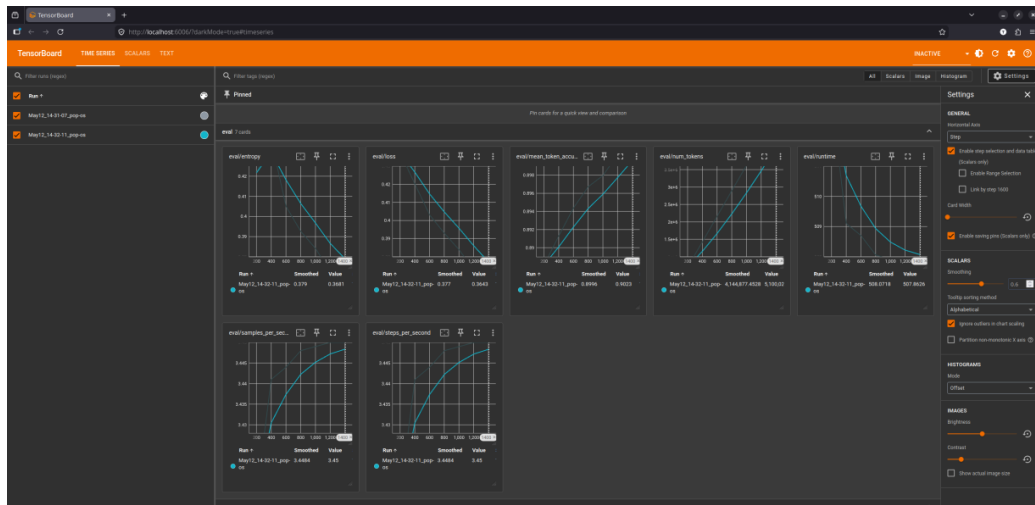
modelin Karışık Uzman (MoE) mimarisinin ince ayar süreçlerini karmaşıktırdığı ve mevcut VRAM bütçesiyle verimli çalışmadığı gözlemlenmiştir.

8.3.2. QLoRA Yöntemi

İnce ayar işlemi, bellek etkin bir parametre adaptasyon tekniği olan QLoRA (Nicelenmiş Düşük Dereceli Uyarlama) yöntemiyle gerçekleştirilmiştir. Bu yöntemde temel model ağırlıkları dondurularak 4-bit nicemleme (quantization) uygulanmakta; yalnızca eklenmiş düşük dereceli adaptör (LoRA adapter) matrisleri eğitilmektedir. Böylece tüm parametreleri güncelleyen tam ince ayara (full fine-tuning) kıyasla belirgin biçimde daha düşük VRAM tüketimiyle eğitim gerçekleştirilebilmektedir. Maksimum dizi uzunluğu mevcut VRAM kısıtları nedeniyle 1.024 token olarak belirlenmiştir. Geliştirilen modelin ince ayar sürecine ait değerlendirme metriği grafiği **Şekil 7**'de verilmiştir. Eğitim Sürecine Ait TensorBoard Metrik Panosu ise **Şekil 8**'de gösterilmiştir.



Şekil 7. İnce Ayar Sürecine Ait Değerlendirme (Evaluation) Metriği Grafiği



Şekil 8. Eğitim Sürecine Ait TensorBoard Metrik Panosu

8.3.3. RISC-V Ajan Veri Seti

Eğitim verisi, AkyuzIDE'nin 'Ajan', 'Sor' ve 'Planla' modlarına özgü davranışları öğretmek amacıyla elle hazırlanmış örneklerden oluşan bir denetimli ince ayar (SFT) veri setidir. Her kayıt şu alanları barındırmaktadır: benzersiz kimlik (id), mod türü (agent/ask/plan), sohbet bağlamı (messages dizisi) ve modelin o turda üretmesi beklenen çıktı (assistant alanı). Kalite kurallarına göre, ajan modundaki kayıtlar yalnızca bir araç çağrısı içermeli ve araç çağrısı JSON biçiminde ayrıştırılabilir olmalıdır; sor modundaki kayıtlar araç çağrısı içermemeli, teknik Türkçe açıklama sunmalıdır; planla modundaki kayıtlar dosya yazma ya da kabuk komutları yerine salt okunur araçları ve plan üretme komutunu kullanmalıdır. Veri seti onaylı araçların dışında araç çağrısı içeren örnekleri doğrulama betiği (validate_dataset.py) aracılığıyla otomatik olarak elemektedir. Türkçe açıklama ile İngilizce teknik terim bileşiminden oluşan karma dil yapısı bilinçli olarak benimsenmiş; modelin hem Türkçe kullanıcı mesajlarını anlaması hem de EDA araçlarıyla teknik terimler üzerinden iletişim kurması hedeflenmiştir.

8.3.4. Eğitim Süreci

Eğitim, 16 GB VRAM kapasiteli NVIDIA GeForce RTX 5070 Ti grafik işlem birimi üzerinde yürütülmüştür. Temel hiperparametreler şu biçimde belirlenmiştir: öğrenme hızı 2×10^{-4} , dereceli ısınma adımı (warmup steps) 50, toplu iş boyutu 1, gradyan birikimi 8 adım. Eğitim döngüsü boyunca modelin Verilog HDL kurallarına uyumu doğrulama kümesi üzerinde değerlendirilmiş; sentezlenemeyen kod üretme oranının taban modele kıyasla belirgin biçimde düştüğü gözlemlenmiştir.

8.3.5. Eğitim Süreci Kıyaslama ve Yayın Geçidi

Eğitim tamamlandıktan sonra model, projeye özgü bir kıyaslama (benchmark) sistemiyle değerlendirilmektedir. Kıyaslama, modelin ALU tasarımı, yazmaç dosyası üretimi, boru hattı tehlike tespiti ve SoC çevresel birim entegrasyonu gibi donanım tasarım görevlerindeki başarısını ölçmektedir. Yayın geçidi (release_gate.py) bu kıyaslama sonuçlarını değerlendirerek belirli bir başarı eşiğini geçemeyen modellerin üretime alınmasını engellemektedir. İnce ayar sürecinin çıktısı olan akyuz-qwen25-coder:14b-q4 modeli, Ollama GGUF biçimine dönüştürülerek yerel çıkarım motoruna yüklenmiştir. Bu model, yürütücü rolünde saniyede seksen iki belirtecin üzerinde çıkarım hızına ulaşmaktadır.

9. UYGULAMA

Projenin birinci aşamasında, açık kaynaklı mikroişlemci tasarımının tüm teorik zorluklarını ve yapıtaşlarını kavramak amacıyla; RV32I temel tam sayı komut setini eksiksiz olarak destekleyen, modern işlemcilerin temelini oluşturan 5 aşamalı ardışık boru hattı mimarisiyle donatılmış gerçek bir RISC-V işlemci çekirdeği Verilog HDL dili ile sıfırdan tasarlanmıştır. Bu donanım tasarımı; boru hattındaki komutların birbiriyle veri çakışmasına girmesini donanımsal yollarla aşan yönlendirme (forwarding) birimini, gecikmeli bellek okumalarında hattı donduran tehlike tespit (hazarddetection) birimini içeren kapsamlı bir çekirdeği barındırmaktadır.

Projenin ikinci ve inovasyon odaklı aşamasında ise, donanım geliştiricilerine ve bu alana yeni adım atan araştırmacılara doğrudan hitap eden, oluşturulan bu RISC-V çekirdeğinin simülasyonunu ve genel Verilog HDL modüllerinin geliştirilmesini otomatize ederek kolaylaştıracak olan bir masaüstü yazılım ortamı geliştirilmiştir.

Genel amaçlı RTL devrelerinin, işlemci donanım modüllerinin ve SoC mimarilerinin verimli ve hatasız kodlanması amacıyla geliştirilen bu masaüstü platformu, asenkron bir yazılım mimarisine sahiptir. Sistem; kullanıcı arayüzü, çekirdek servisler ve harici bağımlılıklar olmak üzere birbirinden bağımsız üç temel katman üzerinde yapılandırılmıştır.

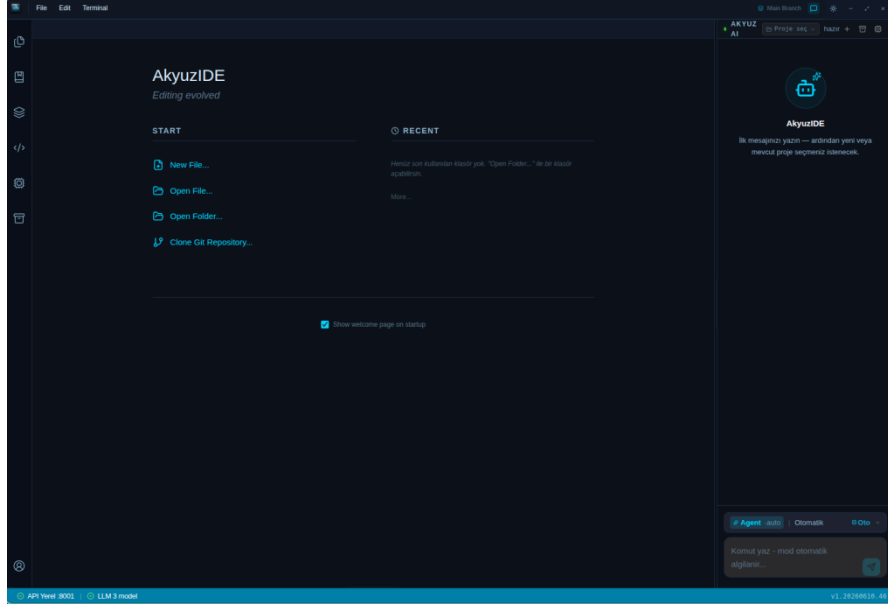
9.1. Kullanıcı Arayüzü

Sistem ilk çalıştırıldığında kullanıcıyı karşılayan kurulum ve altyapı kontrol süreçleri **Şekil 9**'da verilmiştir.



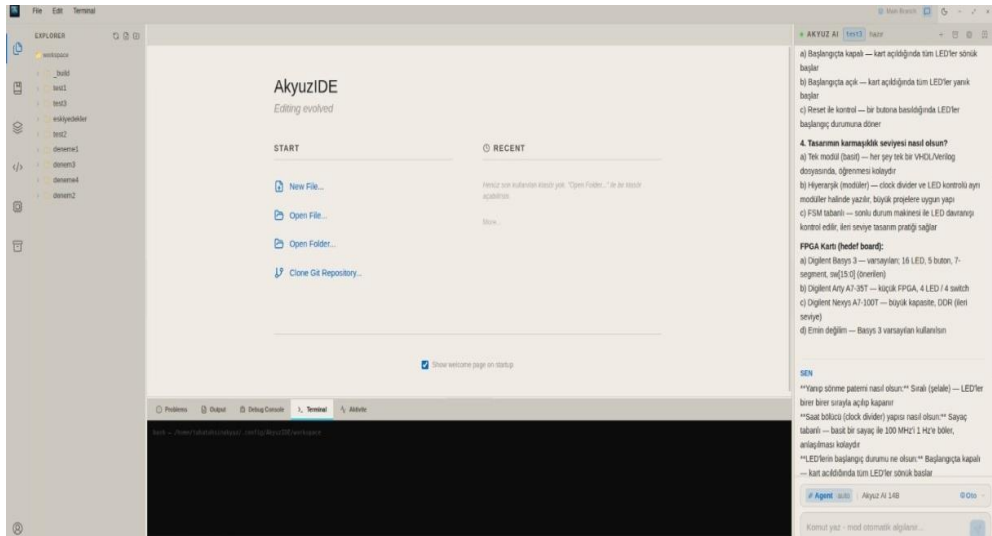
Şekil 9. AkyuzIDE Kurulumu ve Sistem Araç Taraması

Kurulum tamamlandıktan sonra, kullanıcının süreç boyunca yapay zeka desteği alabileceği arayüz bileşeni ise **Şekil 10**'da gösterilen sohbet paneli açık hali ile sunulmaktadır.



Şekil 10. AkyuzIDE Sohbet Paneli Açık Hali

React 19, TypeScript ve Electron ile geliştirilen masaüstü uygulamasının arayüzü, Vite ve Tailwind CSS tabanlı üç ana panelden oluşur. Sol panelde hiyerarşik dosya gezgini (ActivityBar) yer alırken; orta panel Verilog/RISC-V Assembly destekli Monaco Editör'ü, terminal konsolunu ve VCD dalga formu görüntüleyicisini barındırır. Sağ panelde ise kullanıcının doğal dille tasarım taleplerini ilettiği yapay zekâ sohbet alanı (ChatPanel) bulunur. **Şekil 11**'de uygulama arayüzü verilmiştir.



Şekil 11. Uygulama Arayüzü

Bunların yanı sıra FPGA donanımı için HardwareSidebar, tasarım sihirbazı (DesignWizard), C derleyici paneli (CCompilerPanel) ve Vivado yönetim paneli (VivadoPanel) gibi özel bileşenler uygulamaya entegre edilmiştir.

Ön uç ile arka uç arasındaki iletişim, api.ts servis katmanı üzerinden yürütülmektedir. Ajan çalıştırma oturumları Sunucu Tarafından Gönderilen Olaylar (SSE) protokolüyle gerçek zamanlı akış hâlinde istemciye iletilmektedir. Her SSE olayı JSON biçiminde yapılandırılmış ve 'narration', 'tool_call', 'tool_result', 'plan_proposal' ile 'done' türlerini barındırmaktadır.

Arka uç, Python FastAPI çerçevesiyle geliştirilmiş eşzamansız (asynchronous) bir web sunucusudur ve 8001 numaralı port üzerinden hizmet vermektedir. Yapay zekâ modeli iletişimini üstlenen ai_remote.py servisi; yerel Ollama uç noktasına (localhost:11434) veya Cloudflare Tunnel üzerinden uzak bir sunucuya HTTP akış istekleri göndermekte, OpenRouter uyumlu API standardını da desteklemektedir.

Simülasyon hizmetini sağlayan simulation.py; Verilog kaynak dosyalarını geçici dizine aktarıp IcarusVerilog derleyicisini alt süreç olarak çalıştırmakta, standart hata ve standart çıktı kanallarını yakalayarak sonuçları analiz etmekte ve isteğe bağlı olarak VCD dalga formu dosyasını üretmektedir. hardware.py servisi pyserial kütüphanesi aracılığıyla USB bağlantı noktalarını tarayarak FTDI ve SiliconLabs VID/PID kimliklerine göre FPGA kartını otomatik olarak tespit etmektedir. assembler.py, herhangi bir harici bağımlılık gerektirmeksizin salt Python ile yazılmış bir RV32I birleştirici (assembler) modülüdür.

9.2. Çekirdek Denetleyiciler Katmanı

Uygulamanın iş mantığını ve otonom karar mekanizmalarını barındıran kontrol omurgasıdır. session_manager.py her projeye özgü oturum durumunu, etkin çalışma alanını ve günlük yalıtımını (log isolation) yönetmektedir. smart_router.py; gelen kullanıcı mesajını, görev türünü ve sistem bellek profilini çapraz değerlendirerek hangi büyük dil modelinin kullanılacağına karar vermektedir. Bu modülde tanımlanan görev türleri şunlardır: 'verilog_write', 'verilog_debug', 'fpga_pipeline', 'complex_design', 'general_ask' ve 'greeting'. Görev türü ve kullanılabilir VRAM miktarına göre en uygun model seçilmektedir.

9.3. Harici Yazılım Bağımlılıkları Katmanı

Projenin açık kaynaklı yazılım ekosistemi şu temel bileşenlerden oluşmaktadır:

- **Ollama:** Büyük dil modellerini (LLM) yerel donanım üzerinde çalıştıran ve optimize eden çıkarım motorudur.

- **IcarusVerilog:** IEEE 1364 standardına uygun olarak çalışan, donanım tanımı doğrulamalarında kullanılan RTL simülasyon aracıdır.
- **RISC-V GCC:** C dilinde geliştirilen uygulamaları, hedef donanıma uygun RISC-V makine koduna dönüştüren çapraz derleyici araç zinciridir.
- **GTKWave:** Simülasyon çıktıları olan VCD (Value Change Dump) dalga formu dosyalarını görsel olarak inceleme imkânı sunan dalga formu analizcisidir.
- **Vivado Design Suite:** FPGA sentezleme, yerleştirme/yönlendirme (place & route) ve donanımın programlanması için bit akışı (bitstream) üretme süreçlerini yürüten endüstriyel EDA aracıdır.

Tasarlanan masaüstü platformunun donanım doğrulama yeteneklerini ve test otomasyonu süreçlerini ölçmek amacıyla yürütülen simülasyonların sonuçları, platformun kullanıcı arayüzünde dinamik olarak raporlanmaktadır. Geliştirilen platformun 14 farklı sınır senaryosunu başarıyla doğruladığı ve elde edilen döngü-senkron analiz özeti **Şekil 12**'de verilmiştir.

ADD, SUB, ADDI	ALU aritmetik doğrulama	✓
SLL, SRL, SRA	Bit kaydırma komutları	✓
LUI, AUIPC	20-bit yükleme & PC aritmetiği	✓
LW, SW (Bellek)	32-bit RAM okuma/yazma	✓
RAW Data Hazard (Forwarding)	Veri bağımlılığı çözümü	✓
Load-Use Stall	1 saat döngüsü duraklatma	✓
Branch BEQ/BNE	Şartlı dal + IF/ID Flush	✓
JAL, JALR	PC atlama & geri dönüş adresi	✓
SLT, SLTU	İşaretili/işaretsiz karşılaştırma	✓

Şekil 12. Donanım Doğrulama Sonuçları

9.4. Alternatif Güvenli Tasarım Akışı ve Otomasyon Modülü

Geliştirilen entegre platform bünyesinde, donanım mühendisliği süreçlerine yön vermek üzere ikinci bir bağımsız tasarım yolu tasarlanmıştır. Bu alternatif yol, donanım mimarisinin kod

üretim sorumluluğunu doğrudan tasarımcıya bırakırken; kalan tüm derleme, simülasyon ve fiziksel doğrulama süreçlerini tek bir çatı altında otomatikleştirmektedir. Söz konusu yapı; özellikle yüksek veri güvenliği, gizlilik ve fikri mülkiyet korunması hassasiyeti bulunan kurumsal Ar-Ge ortamlarına hitap etmektedir. Stratejik öneme sahip donanım kodlarının, veri güvenliği politikaları veya gizlilik protokolleri gereği yerel ya da bulut tabanlı yapay zekâ modelleri tarafından işlenmesini kesinlikle tercih etmeyen kıdemli geliştiriciler ve kurumlar için bu modül, tamamen izole bir çalışma mimarisi sunmaktadır.

Tasarım akışı, kullanıcının kendi geliştirdiği veya doğruluğu onaylanmış sentezlenebilir ham Verilog HDL kodlarını platform üzerindeki entegre editör arayüzüne manuel olarak girmesiyle başlamaktadır. Sisteme yüklenen kullanıcı kodları, herhangi bir yapay zekâ süzgecine uğramadan veya uzak sunuculara aktarılmadan, tamamen yerel (offline) çalışan kurallı otomasyon betikleri tarafından işlenmektedir. İlk adımda, girilen kodların statik analizi ve sentaks kontrolleri yapılarak olası yazım hataları kullanıcıya listelenmekte; hemen ardından kodlar otomatik olarak simülasyon katmanına aktarılmaktadır. Bu süreçte geliştirici; endüstriyel elektronik tasarım otomasyonu (EDA) araçlarının karmaşık arayüzlerini manuel manipüle etmek, ağır simülatör konfigürasyonlarıyla zaman kaybetmek veya uzun TCL komut dosyaları yazmak zorunda kalmamaktadır. Entegre edilen otomasyon motoru; simülasyon testlerini, zamanlama analizlerini, sentez (synthesis) ve fiziksel gerçekleştirme (implementation) adımlarını arka planda çalışan özelleştirilmiş betikler vasıtasıyla tek bir tıklama ile ardışık olarak icra etmektedir. Sürecin sonunda üretilen bitstream dosyaları, sentez raporları ve dalga şekli (waveform) analiz çıktıları doğrudan arayüze entegre edilerek kullanıcıya sunulmaktadır.

Sonuç olarak hazırlanan bu ikinci tasarım yolu, yapay zekâ kaynaklı veri sızıntısı risklerini tamamen ortadan kaldırırken, geleneksel donanım tasarım araçlarının hantal ve dağınık süreçlerini otomatikleştirerek kullanıcıya güvenli, yerel ve yüksek verimlilikli bir tasarım alternatifi sağlamaktadır.

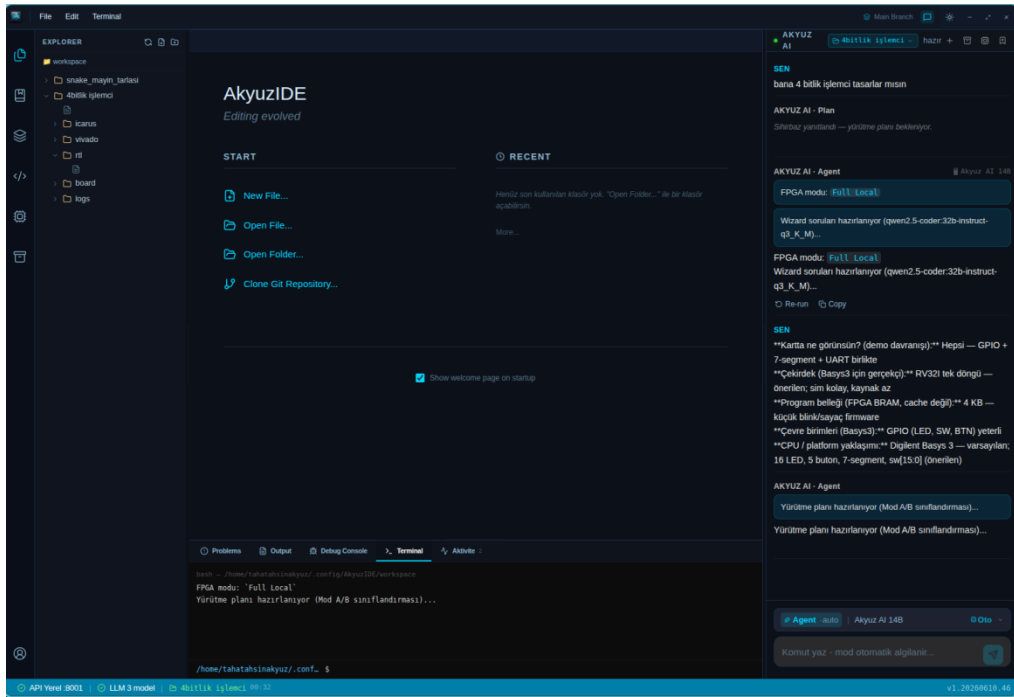
10. AJAN MOTORU

AkyuzIDE' nin temel yeniliği, donanım tasarım döngüsünü uçtan uca yöneten otonom ajan motorudur. Motor, 'Reason + Act' (ReAct) felsefesine dayalı olarak çalışmakta ve altı katmanlı bir mimari üzerine inşa edilmiş bulunmaktadır.

10.1. Planlayıcı ve Yürütücü Model

Ajan motoru, görev karmaşıklığını en iyi şekilde karşılayacak modeli seçebilmek amacıyla Planlayıcı (Planner) ve Yürütücü (Executor) olmak üzere iki farklı model rolü tanımlamaktadır.

Planlayıcı rol, kullanıcının doğal dildeki isteğini çözümleyerek yapılandırılmış bir JSON yürütme planı üretmek için derin muhakeme gerektiren karmaşık görevlerde devreye girmektedir; bu rol için deepseek-r1:14b veya qwen3:14b modelleri tercih edilmektedir. Yürütücü rol ise araç çağrısı döngüsünü hızlı biçimde sürdürmek amacıyla projeye özel ince ayar yapılmış akyuz-qwen25-coder:14b-q4 modelini kullanmaktadır. Bu iki rol arasındaki iş bölümünün temel gerekçesi, hem bulut tabanlı modelin güçlü muhakeme kapasitesinden hem de yerel modelin yüksek işlem hızından eş zamanlı yararlanmaktır. Bulut tabanlı planlama yalnızca oturum başında bir kez gerçekleştirilirken, sonraki tüm araç çağrısı döngüsü yerel model üzerinden sürdürülerek sistem performansı optimize edilir. AkyuzIDE ajan modu arayüzü **Şekil 13**’de gösterilmiştir.



Şekil 13. AkyuzIDE Ajan Modu

Maksimum 60 turla sınırlandırılan temel çalışma döngüsü, her aşamada belirli bir işlem sırasını takip eder. İlk olarak geçmiş mesajlar ve aktif araç şemaları modele iletilir; ardından gelen yanıt Qwen3 yerel biçimi, XML-A, XML-B ve JSON olmak üzere dört farklı araç çağrısı sözdizimi için çözümlenir. Geçerli bir araç çağrısı tespit edildiğinde öncelikle güvenlik kapıları tarafından denetlenir, onaylanan araç çalıştırılır ve elde edilen sonuç bir sonraki turun girdisi olarak sisteme dahil edilir.

10.2. Deterministik EDA Boru Hattı ve Güvenlik Kapıları

Donanım tasarımı araç çağrısı sırasının kritik önem taşıması nedeniyle ajan motoruna deterministik bir yürütme kanalı (pipeline) ve zorunlu güvenlik kapıları eklenmiştir. Bu güvenlik kapıları şu kuralları uygulamaktadır: simülasyon geçmeden Vivado sentezi başlatılamaz; bit akışı dosyası üretilmeden FPGA programlama komutu verilemez; kullanıcı tarafından onaylanmış bir plan olmadan dosya yazma işlemi gerçekleştirilemez. Bu mekanizma, modelin hatalı sıralamalardan kaynaklanan geri dönülemez hataların önüne geçmektedir.

10.3. Otonom Onarım Döngüsü

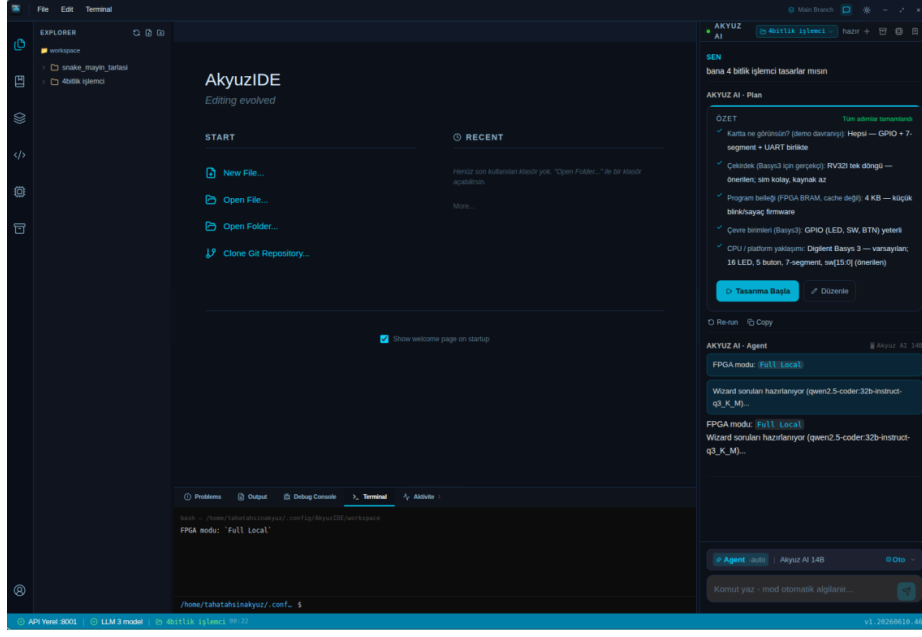
Derleme veya simülasyon başarısız olduğunda otonom onarım döngüsü devreye girmektedir. Başarısız testin dalga formu çıktısı ve derleyici günlükleri modele gönderilmekte; model sinyaller arasındaki ilişkiyi çözümleyerek sorunun kökenini tespit etmektedir. Ardından hatalı Verilog modülü yeniden yazılmakta ve derleme/simülasyon tekrarlanmaktadır. Bu döngü en fazla iki tur sürmekte; iki turda da başarı sağlanamazsa işlemci tasarımının temel işlevselliğini doğrulayan hazır bir şablon devreye alınmaktadır. Döngünün önemli bir özelliği, tüm bu sürecin kullanıcıya yansıtılmadan otomatik olarak tamamlanmasıdır.

10.4. Döngü Tespiti Ve Engel Mekanizması

Aynı dosyaya yapılan yinelenen yazma işlemlerini ve ilerlemesiz araç çağrısı döngülerini engellemek amacıyla motor, her araç çağrısını anahtar-bağımsız değişken karma değeriyle takip etmektedir. Aynı yol için dört kez yazma işlemi gerçekleştiğinde, söz konusu araç kalıcı olarak engellenmekte; art arda üç tur hiçbir ilerleme kaydedilmezse oturum kullanıcıya bir özet raporu sunarak güvenli biçimde sonlandırılmaktadır.

10.5. Plan Onayı (Kullanıcı Gözetimi)

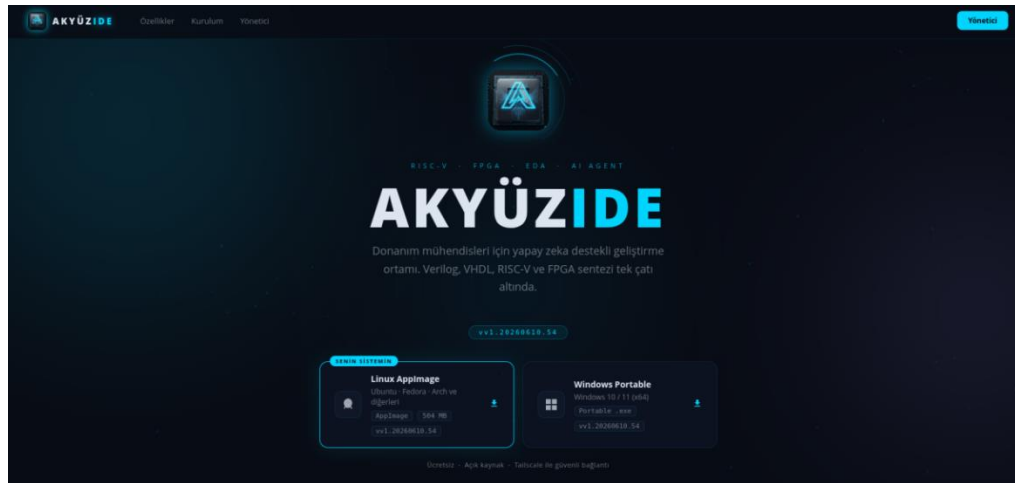
Karmaşık tasarım görevlerinde ajan motoru yürütme öncesinde kullanıcıya bir plan sunmaktadır. Bu plan; proje adı, hedef kart, RTL modüllerinin listesi, tahmini aşamalar ve olası uyarıları içeren yapılandırılmış bir JSON belgesidir. Kullanıcı planı onaylayana dek dosya yazma ve derleme araçları devre dışı kalmaktadır. Bu yaklaşım, bir yandan kullanıcının tasarım sürecinin neresinde olduğunu her zaman görmesini sağlarken öte yandan spesifikasyon sapmasının (spec drift) önüne geçmektedir. Ajan motoru tarafından hazırlanan kodlama stratejisinin kullanıcı denetimine sunulduğu yürütme planı ve onay ekranı **Şekil 14'** de gösterilmiştir



Şekil 14. Yürütme Planı ve Onay Ekranı

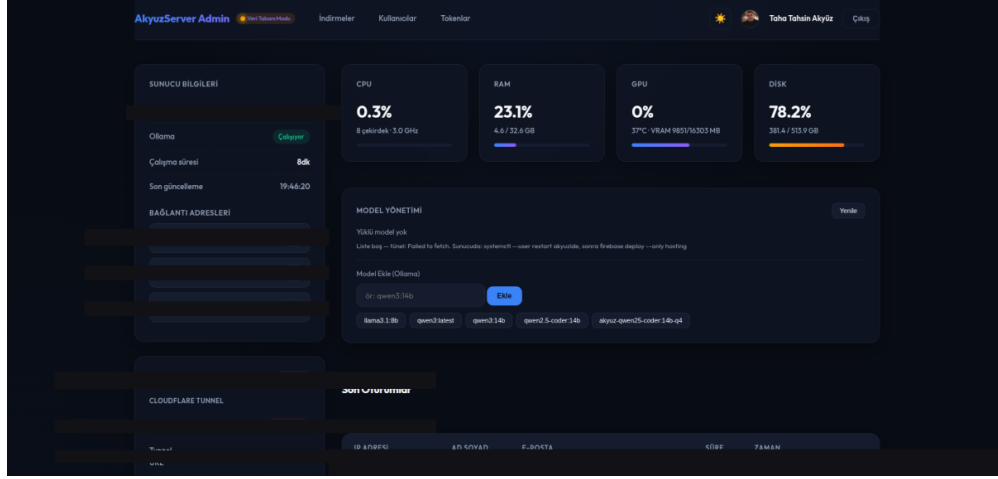
10.6. Sürekli Entegrasyon ve Otomatik Dağıtım

AkyuzIDE'nin yazılım kalitesi ve yayın süreci GitHub Actions ile otomatize edilmiştir. Her main dalı (branch) gönderimi bir yayın iş akışını tetiklemektedir. Bu iş akışı sırasıyla şu adımları çalıştırmaktadır: bağımlılıkların yüklenmesi, RISC-V GCC xpack araç zincirinin ve IcarusVerilog simülatörünün uygulama paketine gömülmesi (bundling), npm run release komutuyla Electron Builder aracılığıyla Linux AppImage ve .deb paketlerinin üretilmesi ve GitHub Releases üzerine otomatik yayımlanması. Proje geliştirme süreci boyunca toplam elli dörtten fazla kararlı sürüm yayımlanmıştır.



Şekil 15. AkyuzIDE Dağıtım Portalı

Uygulamanın dağıtım portalı, sürüm indirme ve platform tespiti **Şekil 15**'te verilmiştir. Bu süreçlerin devamında, gerçek zamanlı sunucu metrikleri ve model yönetimi işlevleri ise **Şekil 16**'da yer alan yönetici paneli üzerinden takip edilmektedir.



Şekil 16. AkyuzIDE Yönetici Paneli

11. İŞLEMCI GELİŞTİRME SÜRECİ

AkyuzIDE' nin işlevselliğini doğrulamak amacıyla sıfırdan geliştirilen RV32I işlemci çekirdeği, dört prototip aşamasından geçmiştir. Her prototip, bir önceki aşamada kazanılan bilgi birikiminin üzerine inşa edilmiş ve yeni teknik hedefler doğrultusunda genişletilmiştir. Bu evrimsel yaklaşım, hem tasarımın olgunlaşmasına hem de IDE' nin her geliştirme aşamasında yeni yeteneklerle sınanmasına imkân tanımıştır

11.1. Temel Çekirdek Doğrulama

RV32I komut setinin temel tamsayı işlemlerini (aritmetik, mantık, dallanma) destekleyen tek çevrimli (single-cycle) bir işlemci modeli geliştirilmiştir. Çalışmanın bu aşamasındaki temel amaç; boru hatlı (pipelining) karmaşıklıklar devreye girmeden önce ALU davranışını, yazmaç dosyası okuma/yazma döngülerini ve kontrol birimi sinyallerini Icarus Verilog simülatöründe doğrulamaktır. İlk aşamayı temsil eden bu model, ilerideki geliştirme süreçlerine temel oluşturan işlevsel bir başlangıç noktası ve modüler bir tasarım altyapısı sağlamıştır.

11.2. Boru Hatlı Çekirdek ve Wishbone SoC

İkinci aşamada tek çevrimli mimari, beş aşamalı boru hatlı (pipeline) bir mimariye dönüştürülmüştür. Boru hattının beraberinde getirdiği veri tehlikeleri (RAW hazards), veri

yönlendirme birimi (forwarding unit) ve tehlike tespit birimi (hazard detection unit) aracılığıyla donanım düzeyinde çözülmüştür. Standart Wishbone B4 boru hatlı veri yolu protokolü benimsenerek çip üstü sistem hiyerarşisi (SoC) kurulmuştur. Tasarıma entegre edilen çevresel birimler şunlardır: UART seri haberleşme birimi, GPIO giriş/çıkış birimi, donanımsal zamanlayıcı (Timer) ve SPI seri çevre birimi. Tamamen simülasyon odaklı olan bu sistem, fiziksel FPGA sentezine yönelik kısıt dosyası (XDC) içermemektedir; doğrulama süreçlerinin tamamı Icarus Verilog ile gerçekleştirilmiştir.

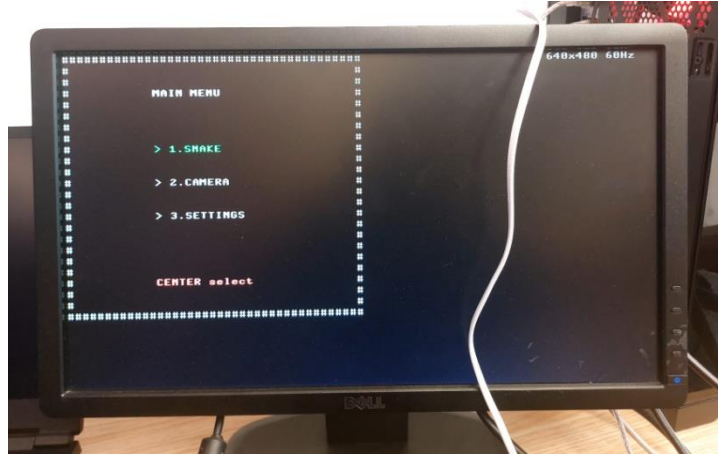
11.3. FPGA Üzerinde Tam SoC — Snake ve Mayın Tarlası

İşlemci tasarımını gerçek FPGA donanımına taşıyan ana geliştirme aşamasında, hedef platform olarak Xilinx Artix-7 (XC7A35T) yonga setini barındıran Digilent Basys 3 geliştirme kartı kullanılmıştır. Sistem saati 100 MHz ana osilatörden elde edilerek uygulamaya göre bölünmektedir; bu doğrultuda işlemci çekirdeği 50 MHz, VGA piksel birimi ise 25 MHz frekansında çalışmaktadır. Vivado 2024.1 sentez raporuna göre en kötü negatif süre farkı (WNS) yaklaşık +5 ns olarak ölçülmüş ve tasarımın zamanlama gereksinimlerini bütünüyle karşıladığı doğrulanmıştır. Sistemin en dikkat çekici yeniliği, 80×30 karakter kapasiteli donanımsal VGA terminalidir. Altyapıda yer alan font_rom modülü 8×16 piksel boyutundaki bitmap karakter verilerini barındırırken, vga_text_ctrl modülü karakter tamponundan okuduğu ASCII kodlarını piksel koordinatlarına göre parlaklık bilgisine dönüştürmektedir. Ayrıca işlemciden bağımsız çalışabilen donanımsal bir plazma animasyon modülü (Graphics Bypass) mimariye dahil edilmiştir. Donanımsal UART önyükleyici (hw_bootloader) sayesinde bit akışı (bitstream) yeniden yüklenmeksizin yalnızca yazılım güncellenebilirken, SCCB protokolüyle sisteme bağlanan OV7670 CMOS kamera modülü ile VGA terminali arasında anlık anahtar (SW15) kontrolüyle canlı görüntü geçişi sağlanabilmektedir.

Geliştirilen bu donanım altyapısı üzerinde yazılım koşturabilmek amacıyla kapsamlı bir C yazılım geliştirme zinciri (toolchain) kurulmuştur. Süreçte riscv-none-elf-gcc çapraz derleyicisi; crt0.S başlangıç kodu ve sisteme özel hazırlanan bağlayıcı betik (linker.ld) ile birlikte C kaynak dosyalarını derlemektedir. Ardından objcopy aracılığıyla saf ikili biçime (.bin) dönüştürülen üretim çıktıları, upload.py betiği yardımıyla UART bootloader üzerinden karta aktarılmakta ve işlemci otomatik olarak sıfırlanarak yeni yazılımla yürütmeye başlamaktadır.

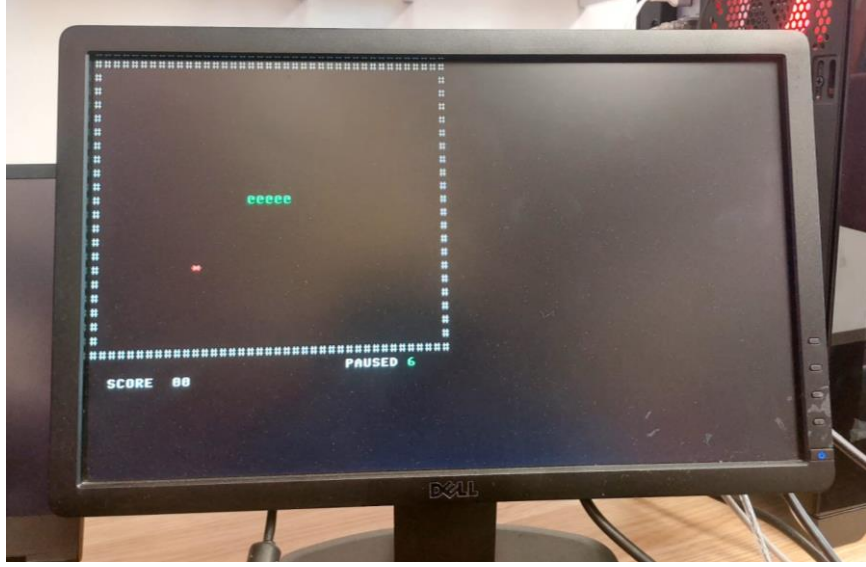
Sistemin işlevselliğini ve donanım-yazılım entegrasyonunu doğrulayan somut uygulamalar ise RV32I işlemcisinin çalıştırdığı Yılan (Snake) ve Mayın Tarlası oyunlarıdır. RISC-V mimarisi için derlenen ve VGA terminali üzerinden ekrana yansıtılan bu oyunlardan Yılan'da; karakter konumları koordinatlarla, hız performansı döngü sayaçlarıyla, yön kontrolleri ise GPIO üzerinden

butonlarla (BTNU/BTND/BTNL/BTNR) idare edilerek skorlar LED'lere aktarılmaktadır. Izgara tabanlı bir bellek modeli kullanan Mayın Tarlası veri belleğindeki (DMEM) matris verisini gerçek zamanlı olarak ekrana taşımaktadır. Bu iki uygulama; program sayacının doğruluğunu, VGA ile CPU'nun paylaşımlı BRAM üzerinde çalışmasını ve GPIO biriminin gecikmesiz kullanıcı etkileşim yeteneğini başarıyla kanıtlamıştır. Sistem ilk açıldığında ekrana gelen ve Snake oyunu, canlı kamera görüntüsü ile sistem ayarları arasında geçiş yapmayı sağlayan terminal ana menüsü **Şekil 17**'de gösterilmiştir.



Şekil 17. VGA Terminal Ana Menüsü

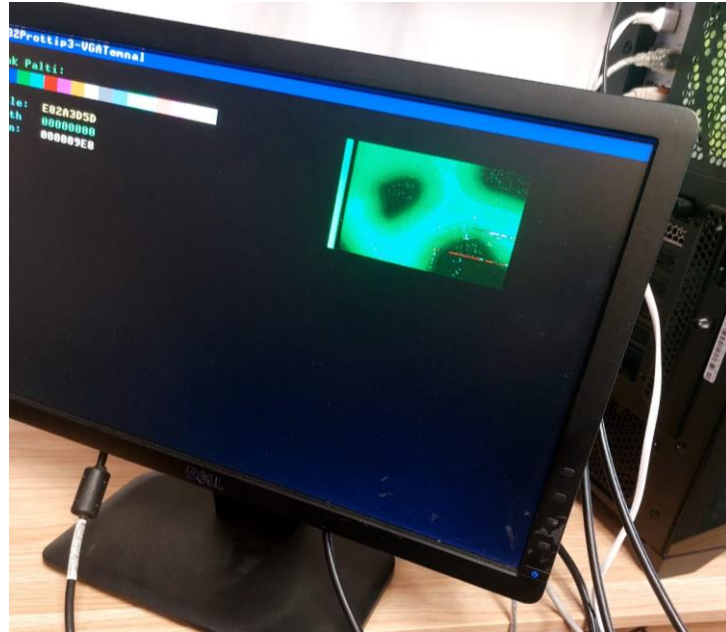
Sistemin işlevselliğini ve donanım-yazılım entegrasyonunu doğrulayan somut uygulamalar ise RV32I işlemcisinin çalıştırdığı Yılan (Snake) ve Mayın Tarlası oyunlarıdır. RISC-V mimarisi için derlenen ve VGA terminali üzerinden ekrana yansıtılan bu oyunlardan Yılan'da; karakter konumları koordinatlarla, hız performansı döngü sayaçlarıyla, yön kontrolleri ise GPIO üzerinden butonlarla (BTNU/BTND/BTNL/BTNR) idare edilerek skorlar LED'lere aktarılmaktadır. Izgara tabanlı bir bellek modeli kullanan Mayın Tarlası veri belleğindeki (DMEM) matris verisini gerçek zamanlı olarak ekrana taşımaktadır. Bu iki uygulama; program sayacının doğruluğunu, VGA ile CPU'nun paylaşımlı BRAM üzerinde çalışmasını ve GPIO biriminin gecikmesiz kullanıcı etkileşim yeteneğini başarıyla kanıtlamıştır. Tasarlanan işlemci mimarisi üzerinde başarıyla yürütülen Yılan oyununun anlık ekran çıktısı **Şekil 18**'de gösterilmiştir.



Şekil 18. RV32I İşlemcisi Üzerinde Çalışan Snake Oyunu

11.4. Kamera /VGA SoC Sadeleştirilmesi

Dördüncü aşama, bir önceki çalışmanın kamera ve VGA altyapısının yeniden yapılandırılmasıyla elde edilmiştir. Bu adımdaki temel amaç, basys3_soc_cam_top üst modülünü sadeleştirerek bağımsız test ve sentezlemeye elverişli, yalın bir tasarım ortaya koymaktır. Gerçekleştirilen bu optimizasyon sayesinde, kamera arayüzü ve VGA görüntüleme modüllerinin birbirini etkilemeden, bağımsız biçimde doğrulanması sağlanmıştır.



Şekil 19. Kamera Arayüzü

Tasarlanan SoC mimarisinin çoklu görev performansını ortaya koyan kamera arayüzü ve VGA terminali eş zamanlı çıkış görüntüsü **Şekil 19**'da sunulmuştur.

12. SONUÇLAR

Bu çalışma kapsamında bilgisayar mimarisi, donanım mühendisliği ve yapay zekâ teknolojilerinin kesişim noktasında yer alan iki büyük entegre sistem başarıyla hayata geçirilmiş ve doğrulanmıştır. Projenin ana çıktısı olan AkyuzIDE ile bu aracın pratikte ürettiği somut ürün olan RV32I işlemci çekirdeği birlikte değerlendirildiğinde, çalışmanın başında belirlenen tüm temel ve yenilikçi hedeflere eksiksiz ulaşıldığı görülmektedir.

Yazılım katmanında geliştirilen AkyuzIDE; çapraz platform destekli bir masaüstü uygulamasıdır. Altyapısında altı katmanlı ajan motoru, Planlayıcı/Yürütücü çift model mimarisi, deterministik EDA boru hattı ve otonom onarım döngüsü barındıran platform, esnek bir çalışma ortamı sunmaktadır. Donanım geliştirme süreçlerindeki söz dizimi ve simülasyon hatalarını en aza indirmeyi hedefleyen bu platform; Icarus Verilog simülasyon entegrasyonu, otonom çalışan ajan döngüsü, FPGA programlama arayüzü ve saf Python tabanlı RISC-V çevirici modülüyle işlevsel hale getirilmiştir. CI/CD otomasyonu aracılığıyla elli dörtten fazla kararlı sürümü yayımlanan uygulama, Linux AppImage ile .deb paketleri biçiminde kamuoyuyla paylaşılmıştır.

Sistemin zekâ katmanını oluşturan model eğitimi sürecinde, Qwen2.5-Coder-14B-Instruct mimarisi kullanılmıştır. Bu mimari; RISC-V ve Verilog HDL'ye özgü hazırlanan denetimli ince ayar veri seti üzerinde QLoRA yöntemiyle eğitilmiştir. Elde edilen akyuz-qwen25-coder:14b-q4 modeli, yürütücü rolünde saniyede seksen iki belirtecin üzerinde çıkarım hızına ulaşmıştır. Çevrimdışı çalışabilen bu yerel büyük dil modeli altyapısı sayesinde, donanım sentezi kurallarına uyum oranı taban modele kıyasla belirgin biçimde artırılmıştır. Sistemin tamamen yerel donanımda çalışması, gizlilik gerektiren ulusal ve kurumsal çip projeleri için kritik bir veri güvenliği avantajı sağlamaktadır.

Donanım katmanında ise açık kaynaklı RISC-V komut seti mimarisine dayalı bir RV32I işlemci çekirdeği ile bellek, UART, GPIO, Timer ve SPI modüllerini kapsayan çip üstü sistem (SoC) hiyerarşisi Verilog HDL ile başarıyla tasarlanmıştır. Dört farklı geliştirme evresinden geçen bu donanımın ikinci aşamasında, Wishbone B4 veri yolu protokolü benimsenerek simülasyon odaklı bir SoC oluşturulmuştur. Tasarımın gerçek FPGA donanımına (Digilent Basys

3, Artix-7) taşındığı üçüncü aşamada ise mimari; 5 aşamalı ardışık düzen (pipeline), veri yönlendirme, tehlike tespit birimleri, 80×30 karakter VGA terminali, UART önyükleyici ve OV7670 kamera arayüzüyle zenginleştirilmiştir. Bu donanım üzerinde C diliyle geliştirilen Snake ve Mayın Tarlası oyunları, RISC-V GCC araç zinciriyle derlenerek UART önyükleyici aracılığıyla karta yüklenmiş ve başarıyla çalıştırılmıştır. Geliştirilen donanımın zamanlama tabanlı simülasyonları ve Xilinx Vivado ortamındaki 14 farklı sınır senaryosu testi tam doğrulukla gerçekleştirilmiştir.

Sonuç olarak bu çalışma; yapay zekâ ile donanım tasarım süreçlerinin yerel, güvenli ve otonom bir çatı altında birleştirilebileceğini kanıtlayan yenilikçi bir mühendislik çözümü sunmaktadır. AkyuzIDE' nin açık kaynak olarak paylaşılması, bilgisayar mimarisi ve dijital donanım tasarımı alanlarındaki akademik çalışmalara ve yerli Ar-Ge kültürüne somut bir katkı sağlamayı hedeflemektedir.

KAYNAKLAR

- [1] "Design & Implementation of 5-Stage 32-bit RISC-V Pipeline Processor on FPGA," in *Proceedings of the 9th MDSRIC*, University of Wah, 2024. Accessed: May 4, 2026. [Online]. Available: <https://www.uow.edu.pk/ORIC/MDSRIC/Publications/9th%20MDSRIC-169.pdf>
- [2] "David Patterson Bibliography - A.M. Turing Award Winner," ACM. Accessed: May 4, 2026. [Online]. Available: https://amturing.acm.org/bib/patterson_2316693.cfm
- [3] D. Patterson and A. Waterman, *The RISC-V Reader: An Open Architecture Atlas*, 1st ed. San Francisco, CA: Strawberry Canyon LLC, 2017. ISBN: 978-0-9992491-1-6.
- [4] H. M. Ahmed, "RV32I 5-Stage Pipelined Processor (SystemVerilog Implementation)," GitHub repository, 2024. Accessed: May 4, 2026. [Online]. Available: <https://github.com/HafizMutahirAhmed/RV32I-5-Stage-Processor>
- [5] A. Jacobo, "RISC-V: Design implementation of the RV32I Core in Verilog HDL with Zicsr extension," GitHub repository, 2023. Accessed: May 4, 2026. [Online]. Available: <https://github.com/AngeloJacob/RISC-V>
- [6] E. Sarıtaş and S. Karataş, *Her Yönüyle FPGA ve VHDL [FPGA and VHDL in Every Aspect]*, 3rd ed. Ankara, Türkiye: Palme Yayıncılık, 2013. ISBN: 978-605-355-129-4.
- [7] Ali-975, "RISC-V RV32I 5-stage pipelined processor implemented in SystemVerilog with RTL design, testbench, and hex-based instruction memory," GitHub repository, 2024. Accessed: May 4, 2026. [Online]. Available: <https://github.com/Ali-975/rv32i-5-stage-pipelined-processor>

- [8] "Andrew Waterman," Google Scholar Citations. Accessed: May 4, 2026. [Online]. Available: https://scholar.google.com/citations?user=jQY3_ncAAAAJ&hl=en
- [9] S. Yadav S., "Design and Implementation of a 5-stage Scalar In-order Pipelined RISC Processor," M.Tech. thesis, Dept. Elect. Eng., Indian Institute of Technology Madras, Chennai, India, 2014.
- [10] F. Karataş, İ. Koyuncu, M. Alçın, and M. Tuna, "II. Derece AV Blok Aritmik EKG Sinyallerinin VHDL ile FPGA-Tabanlı Tasarımı" [FPGA-Based Design of Grade II AV Block Arrhythmic ECG Signals with VHDL], *Afyon Kocatepe Üniversitesi Fen ve Mühendislik Bilimleri Dergisi*, vol. 22, no. 6, pp. 1334-1345, Dec. 2022. DOI: 10.35414/akufemubid.1141837.
- [11] "RISC-V Instruction Sets," Devopedia, 2021. Accessed: May 4, 2026. [Online]. Available: <https://devopedia.org/risc-v-instruction-sets>
- [12] "SiliconCompiler Project Documentation," SiliconCompiler. Accessed: May 4, 2026. [Online]. Available: <https://www.siliconcompiler.com>
- [13] M. B. Erdem, *VHDL ve FPGA ile Dijital Sistem Tasarımı* [Digital System Design with VHDL and FPGA], 1st ed. Ankara, Türkiye: Seçkin Yayıncılık, 2022. ISBN: 978-605-264-154-5.
- [14] "A Novel 32 Bit RISC-V Based Pipelined Processor Design Using Verilog," B.E. Project Report, SATHYABAMA Institute of Science and Technology, Chennai, India.
- [15] Aakash A., Aparna R., A. B. Abraham, and S. R. Malathi, "Design of 5 Stage Pipelined RISC-V Processor," *International Journal of Innovative Research in Technology (IJIRT)*, vol. 10, no. 12, pp. 2779-2783, May 2024. ISSN: 2349-6002.
- [16] Varunkumar0610, "RISC-V-32I-5-stage-Pipeline-Core," GitHub repository, 2022. Accessed: May 4, 2026. [Online]. Available: <https://github.com/Varunkumar0610/RISC-V-32I-5-stage-Pipeline-Core>
- [17] *IEEE Standard for Verilog Hardware Description Language*, IEEE Std. 1364-2005 (Revision of IEEE Std. 1364-2001), 2006. DOI: 10.1109/IEEESTD.2006.99495.
- [18] "NexusV: A RISC-V based SoC on FPGA," Virtual Expo, Corpus - IEEE NITK, 2021. Accessed: May 4, 2026. [Online]. Available: https://ieee.nitk.ac.in/virtual_expo/report/145
- [19] *Basys 3 Reference Manual*, Digilent Reference, 2016. Accessed: May 4, 2026. [Online]. Available: <https://digilent.com/reference/programmable-logic/basys-3/reference-manual>
- [20] "First Contact: Hello World!," BoxLambda, 2022. Accessed: May 4, 2026. [Online]. Available: <https://epsilon537.github.io/boxlambda/hello-world/>
- [21] "PFE report: LearnV lowRISC Project," Université Grenoble Alpes, TIMA Laboratory, 2020.
- [22] A. Hossein, "VeeRwolf (a platform for the VeeR family of RISC-V cores) for Nexys Video Board," GitHub repository, 2023. Accessed: May 4, 2026. [Online]. Available: <https://github.com/chipsalliance/VeeRwolf>

- [23] "UETRV_Pcore: Design Document," UETRV-PCore Documentation, Read the Docs, 2022. Accessed: May 4, 2026. [Online]. Available: https://uetrv-pcore-doc.readthedocs.io/en/main/design_document/
- [24] K. Dikmen, "V-FRONT: Five-stage 32-bit RISC-V CPU in Verilog (Educational, RV32I 2.0 Compliant, Zicsr, Zifencei)," GitHub repository, 2026. Accessed: May 4, 2026. [Online]. Available: <https://github.com/kagandikmen/V-FRONT>
- [25] "5 Stage Pipelined RISC-V Processor in RTL," *ResearchGate Technical Report*, Apr. 2022. DOI: 10.13140/RG.2.2.31821.10724.
- [26] "C & RV32I CPU with Peripherals," Institute of Applied Information Processing and Communications (IAIK), TU Graz, 2019.
- [27] RISC-KC, "basic_rv32s: Instructional RISC-V processor design framework: single-cycle to 5-stage pipeline with FPGA verification," GitHub repository, 2023. Accessed: May 4, 2026. [Online]. Available: https://github.com/RISC-KC/basic_rv32s
- [28] YosysHQ, "picorv32: A Size-Optimized RISC-V CPU," GitHub repository, 2020. Accessed: May 2026. [Online]. Available: <https://github.com/YosysHQ/picorv32>
- [29] S. Shahabi, S. Niar, and A. Ahmadi, "A High-Performance FPGA-Based Hardware/Software Co-Design for Real-Time Biomedical Applications," in *Proceedings of the 2019 IEEE International Conference on Computer Design (ICCD)*, Abu Dhabi, United Arab Emirates, 2019, pp. 626-629. DOI: 10.1109/ICCD46524.2019.00092.
- [30] "SiFive E-Series Embedded Processors," SiFive Cores, 2024. Accessed: May 2026. [Online]. Available: <https://www.sifive.com/cores>
- [31] McKinsey & Company, "The state of AI in 2024: Gen AI adoption spurts and starts to generate value," McKinsey QuantumBlack, May 2024. Accessed: May 2026. [Online]. Available: <https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-state-of-ai>
- [32] Digilent, "What is an FPGA?," Digilent Blog, 2021. Accessed: May 2026. [Online]. Available: <https://digilent.com/blog/what-is-an-fpga/>
- [33] A. Zulfiqar, "5 Stages of Pipeline in Computer Architecture," Medium, May 19, 2024. Accessed: May 2026. [Online]. Available: <https://medium.com/@aylia.zulfiqar29/5-stages-of-pipeline-in-computer-architecture-dc9fca11784e>
- [34] Ş. S. Şadi, "Borulama (Pipelining)," Bilgisayar Kavramları, Nov. 24, 2008. Accessed: May 2026. [Online]. Available: <https://bilgisayarkavramlari.com/2008/11/24/borulama-pipelining/>

STANDARTLAR ve KISITLAR FORMU
Projenin hazırlanmasında uyulan standart ve kısıtlarla ilgili olarak, aşağıdaki soruları cevaplayınız.
1. Projenizin tasarım boyutu nedir? (Yeni bir proje midir? Var olan bir projenin tekrarı mıdır? Bir projenin parçası mıdır? Sizin tasarımınız proje toplamının yüzde olarak ne kadarını

oluşturmaktadır?)
Geliştirilen çalışma, donanım tasarım süreçlerini yapay zekâ desteğiyle otonomlaştıran yeni ve özgün bir projedir. Dünya genelinde büyük teknoloji şirketleri ve sınırlı sayıda öncü araştırma merkezi tarafından benzer çalışmalar yürütülüyor olsa da bu çözümler genellikle ticari kapalı devre sistemlerden oluşmaktadır; son kullanıcı veya bağımsız mühendisler için ulaşılması zordur ve açık kaynaklı değildir. Geliştirilen AkyuzIDE platformu, bu yeteneği tamamen yerel (offline), erişilebilir ve açık kaynaklı bir ekosistem olarak sunduğu için yenilikçidir. Sistem mimarisi, yerel yapay zekâ optimizasyonları ve boru hatlı RISC-V işlemci tasarımının tamamı proje grubu tarafından gerçekleştirildiğinden özgün tasarım oranı %100'dür
2. Projenizde bir mühendislik problemini kendiniz formüle edip, çözdünüz mü? Açıklayınız.
Evet, çözülmüştür. Genel amaçlı büyük dil modellerinin (LLM) donanım tanımlama dillerinin (HDL) eşzamanlı doğasını anlayamayarak "sentezlenemez ve hatalı kod üretmesi" temel mühendislik problemi olarak ele alınmıştır. Dünyadaki dev şirketlerin devasa bütçeli sunucularla çözmeye çalıştığı bu problem, daha kısıtlı kaynaklarla; donanıma özel veri setleriyle yerel modele ince ayar (fine-tuning) yapılarak ve derleyiciden (Vivado) alınan hataları otonom yakalayan bir "otonom hata düzeltme döngüsü" kurularak tamamen yerel bilgisayar ortamında formüle edilmiş ve başarıyla çözülmüştür.
3. Önceki derslerde edindiğiniz hangi bilgi ve becerileri kullandınız?
Projede; Mantıksal Devre Tasarımı (ardışıl devreler), Bilgisayar Mimarisi ve Organizasyonu (RISC-V, pipeline, hazard yönetimi), Yapay Zekaya Giriş / Doğal Dil İşleme (LLM fine-tuning, istem mühendisliği), Nesneye Yönelik Programlama / Yazılım Mühendisliği (AkyuzIDE Python arayüzü ve ajan mimarisi) ile Mikroişlemciler (FPGA entegrasyonu) derslerinde edinilen bilgi ve beceriler aktif olarak kullanılmıştır.
4. Kullandığınız veya dikkate aldığınız mühendislik standartları nelerdir? (Proje konunuzla ilgili olarak kullandığınız ve kullanılması gereken standartları burada kod ve isimleri ile sıralayınız).
▪ IEEE 1364-2005 (Verilog Hardware Description Language): Sentezlenebilir kodlama standartlarına uyum için donanım tanımlama dili standardı dikkate alınmıştır. ▪ RISC-V RV32I (Unprivileged ISA Standard): İşlemci çekirdeğinin komut kümesi mimarisine uyumluluğu için RISC-V International standardı esas alınmıştır. ▪ IEEE 754: Sistemde ihtiyaç duyulan aritmetik işlemlerin standartlaştırılması için kullanılmıştır. ▪ IEEE 1149.1 (JTAG): Basys 3 FPGA kartına otonom program yükleme ve donanımsal hata ayıklama süreçleri için temel alınmıştır.
5. Kullandığınız veya dikkate aldığınız gerçekçi kısıtlar nelerdir? Lütfen boşlukları uygun yanıtlarla doldurunuz.
a) Ekonomi
Büyük şirketlerin kullandığı yüksek maliyetli bulut APT'leri yerine, operasyonel maliyeti sıfıra indiren ve açık kaynaklı yerel (offline) bir LLM entegre edilerek ekonomik kısıtlar aşılmıştır.
b) Çevre sorunları

Büyük veri merkezlerinin yüksek karbon ayak izi üretmesi kısıtına karşı, projenin optimize edilerek yerel tüketici donanımlarında çalıştırılmasıyla enerji tasarrufu sağlanmıştır.
c) Sürdürülebilirlik
Modüler ve çoklu ajanlı yapısı sayesinde, sistemin ileride VHDL veya Chisel gibi farklı donanım dillerini de destekleyecek şekilde genişletilebilmesi sağlanarak sürdürülebilirlik korunmuştur.
d) Üretebilirlik
Üretilen kodların endüstriyel sentez araçları (Vivado) ile tam uyumlu olması ve fiziksel bir Basys 3 FPGA kartı üzerinde gerçek zamanlı çalışabilmesi sağlanarak fiziksel üretebilirlik kısıtı yerine getirilmiştir.
e) Etik
Açık kaynaklı model ve veri setlerinin lisanslarına tam uyum sağlanmış, akademik dürüstlük ve IEEE Etik Kuralları katı bir şekilde gözetilmiştir.
f) Sağlık
Laboratuvar ve bilgisayar başında uzun süreli çalışmaya bağlı oluşabilecek olumsuz etkilere karşı ergonomi kuralları dikkate alınmıştır.
g) Güvenlik
Sektördeki dev firmaların kendi özel tasarımlarını (fıkri mülkiyet/IP) bulut modellerine gönderirken yaşadığı veri sızıntısı ve gizlilik endişeleri, sistemin internete tamamen kapalı (offline) çalışması kısıtıyla aşılmış ve %100 veri güvenliği sağlanmıştır.
h) Sosyal ve politik sorunlar
Yapay zekânın mühendislerin yerini alması kaygısının aksine, bu çalışmayla dünyadaki erişimi zor ve pahalı teknolojilerin demokratikleştirilmesi ve her mühendisin otonom asistanı haline getirilmesi vizyonu benimsenmiştir.